



ОБУЧЕНИЕ БЕСПЛАТНО

Новости ▾

Linux ▾

DevOps / DevNo 🔍 ▾

Введение в AWK: руководство для начинающих

Введение в AWK: руководство для начинающих

Рубрики: [Linux](#) Метки: [awk](#), [linux](#)



Команды LINUX «от А до Z» настольная книга с примерами

Обзор с примерами в формате PDF-книги всегда с под рукой.
Работать с Linux станет гораздо проще!

[СКАЧАТЬ БЕСПЛАТНО](#)

Скриптовый язык **AWK** — это мощный инструмент командной строки, предназначенный для **обработки текста**. Он позволяет быстро и



тех, кто не поленился приложить немного усилий, он может стать незаменимым инструментом при решении широкого спектра задач.

В этой статье мы рассмотрим основные принципы языка сценариев **AWK** и покажем **33 примера использования**, начиная с простых и заканчивая более сложными задачами.

Содержание:

1. Как AWK обрабатывает текст

1. Встроенные переменные AWK

2. Как использовать язык сценариев AWK

1. Вывод заданных строк из файла

1. Вывод всех строк на экран

2. Вывод определенного диапазона строк

3. Вывод всех строк, которые содержат хотя бы одно поле

4. Вывод всех строк, которые содержат хотя бы одно поле или только пробелы

2. Извлечение столбцов из файла

1. Вывод на экран нескольких столбцов

2. Вывод на экран нескольких столбцов с помощью BEGIN

3. Математические операции в столбцах

4. Подсчет количества непустых строк

5. Подсчет количества полей, которые содержат только числа

3. Использование массивов в AWK



2. Определение повторяющихся строк с использованием AWK
 3. Игнорирование повторяющихся строк
 4. Использование временных переменных FS и OFS
 1. Изменение разделителей полей
 2. Удаление лишних пробелов
 3. Отображение столбцов в одну строку
 5. Форматирование полей
 1. Преобразование вывода в таблицу
 2. Операции с числами с плавающей запятой
 6. Использование строчных функций в AWK
 1. Преобразование строки в верхний регистр
 2. Изменение части строки
 3. Разделение полей на подполя
 7. Использование внешних команд в AWK
 1. Добавление даты в файл
 2. Изменение внешних переменных
 3. Вызов динамически сгенерированных команд
3. Выводы

Как AWK обрабатывает



Прежде чем перейти к практической части, давайте разберем как **AWK** работает с содержимым документа. Скриптовый язык при обработке строк текста разбивает каждую из них на **поля**. По умолчанию **разделителем полей** является пробел или табуляция, но этот разделитель может быть изменен пользователем. Обращение к полям осуществляется с помощью **переменных** `$1` , `$2` , `$3` , которые соответствуют первому, второму, третьему и т. д. полям текущей строки.

Например, если у нас есть строка

`Sedicom is the best university` , то `$1` будет содержать слово `Sedicom` , `$2` – `is` , `$3` – `the` и т. п. Поля могут использоваться для выполнения различных операций, таких как сравнение, изменение и объединение содержимого файла.

Встроенные переменные AWK

Язык сценариев поддерживает множество **встроенных переменных**, а также позволяет создавать пользовательские. Эти переменные могут использоваться в скриптах **AWK** без объявления и применяться для хранения информации о текущем поле, количестве обработанных строк и других данных.

Рассмотрим несколько наиболее часто используемых встроенных переменных **AWK**:



- OFS – разделитель вывода (по умолчанию – пробел);
- RS – разделитель записей (по умолчанию – символ новой строки);
- ORS – разделитель вывода записей (по умолчанию – символ новой строки);
- NF – указывает количество полей в текущей записи;
- NR – представляет номер текущей обрабатываемой строки;
- \$0 – указывает содержимое текущей записи целиком;
- \$1 , \$2 , \$3 , ... – содержимое первого, второго, третьего и т.д. полей в текущей записи.

Эти переменные мы и будем использовать для работы с языком **AWK**.

Как использовать язык сценариев **AWK**

Как мы уже говорили, **AWK** активно используется для обработки текстов. Поэтому, для демонстрации Вам его возможностей, создадим два файла с таблицами, которые содержат определенные неформатированные данные.



ОБУЧЕНИЕ БЕСПЛАТНО

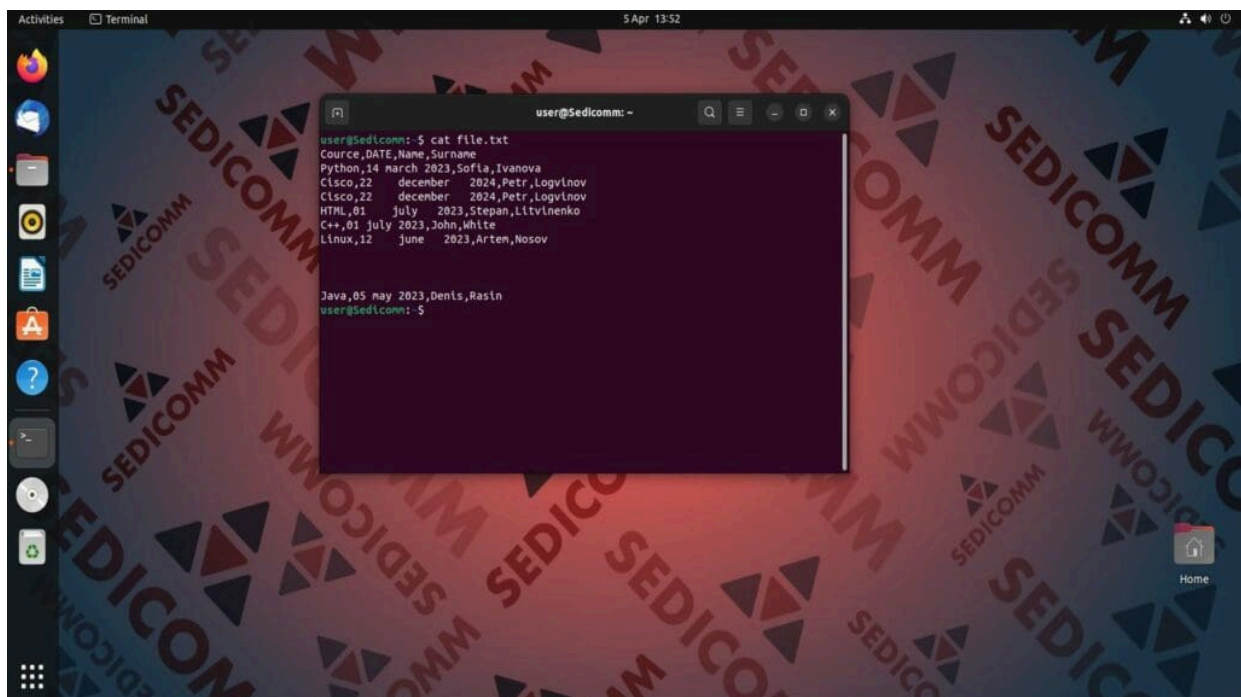
Новости ▾

Linux ▾

DevOps / DevOps 🔍 ▾

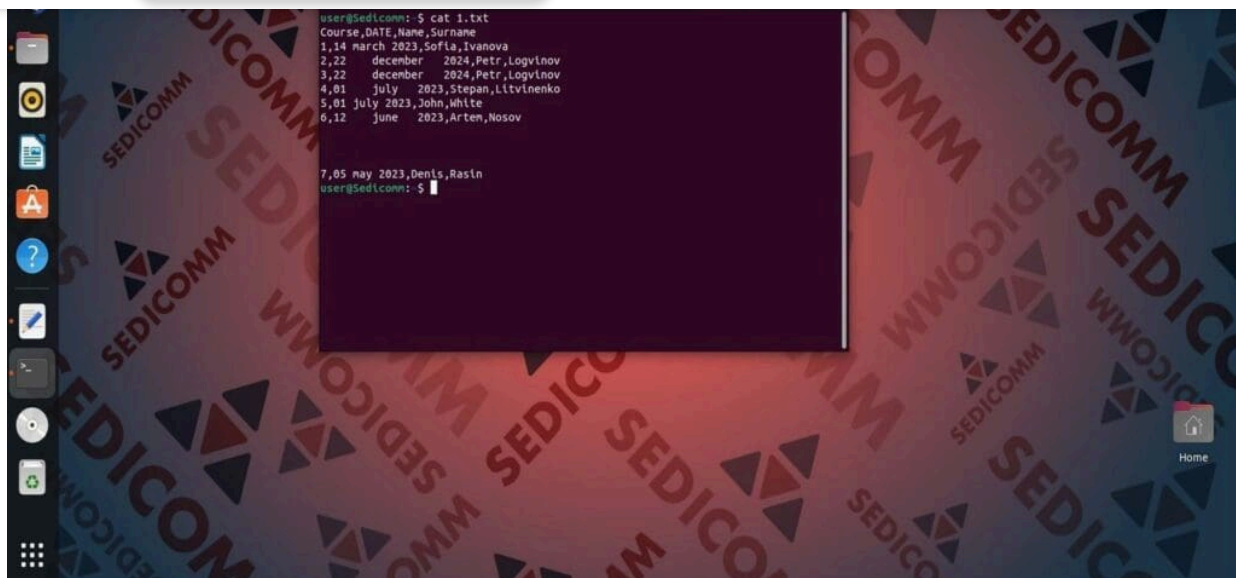
Первый файл:

```
1 cat file.txt
```



И второй файл:

```
1 cat 1.txt
```



```
user@sedicomm: $ cat 1.txt
Course,DATE,Name,Surname
1,14 march 2023,Sofla,Ivanova
2,22 december 2024,Petr,Logvinov
3,22 december 2024,Petr,Logvinov
4,01 july 2023,Stepan,Litvinenko
5,01 july 2023,John,White
6,12 June 2023,Arten,Nosov

7,05 may 2023,Denis,Rasin
user@sedicomm: $
```

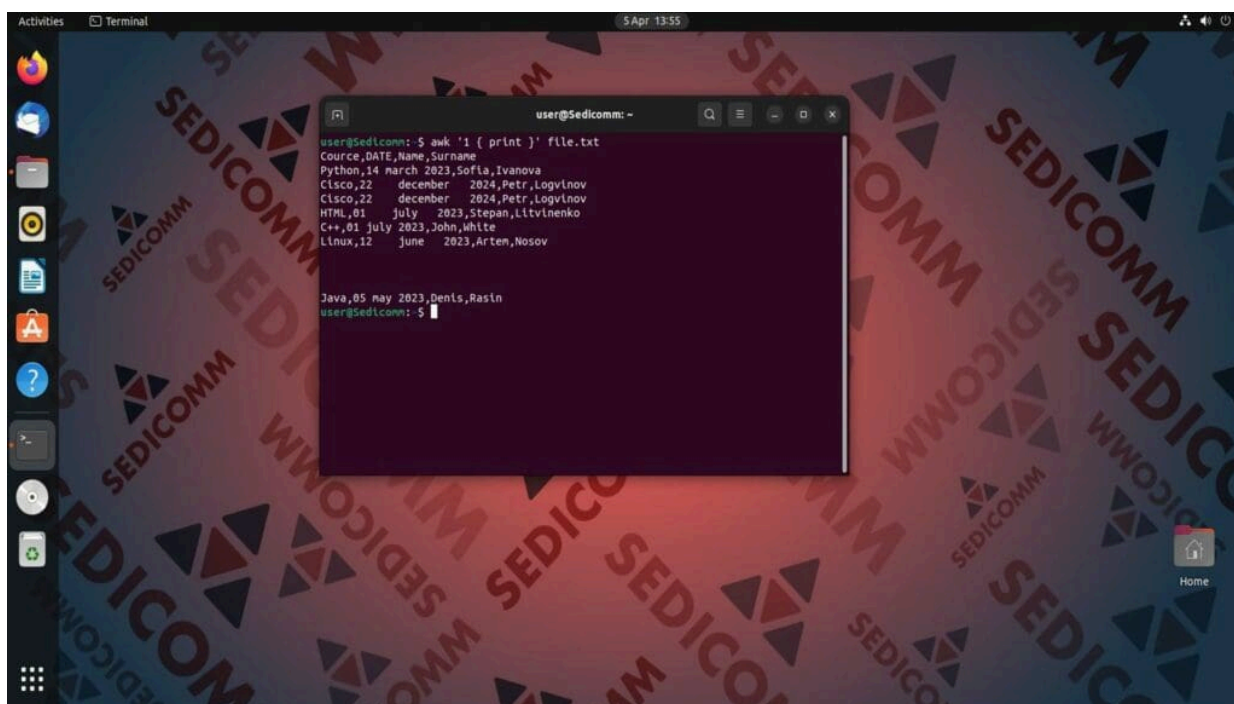
Важно: в обоих файлах есть пустые строки и лишние пробелы. Они будут использоваться для того, чтобы показать отдельные функции языка.

Вывод заданных строк файла

AWK позволяет отображать в терминале **Linux** заданные строки из файла. Для **вывода определенных строк** можно применять специальные фильтры или некоторые временные переменные, которые выбирают нужные строки по определенным критериям. Эта функция очень полезна при обработке файлов с большим количеством строк.

Вывод всех строк на экран

В операционной системе **Linux** существует большое количество команд или утилит, которые позволяют **выводить все содержимое файла на экран**. Данный скриптовый язык также предоставляет пользователям такую возможность. Для того, чтобы вывести все строки файла с помощью **AWK**, используйте следующую команду:

A screenshot of a Linux terminal window. The prompt is 'user@Sedcomm: ~'. The command entered is 'awk '1 { print }' file.txt'. The output is a list of records from a file, each containing a source, date, name, and surname. The records are: Python, 14 march 2023, Sofla, Ivanova; Cisco, 22 december 2024, Petr, Logvinov; Cisco, 22 december 2024, Petr, Logvinov; HTML, 01 july 2023, Stepan, Litvinenko; C++, 01 july 2023, John, White; Linux, 12 june 2023, Arten, Nosov; Java, 05 may 2023, Denis, Rasin. The prompt is now 'user@Sedcomm: ~'.

В **AWK** каждая строка файла воспринимается как запись. В этом случае, поскольку условие (шаблон) `1` соответствует истине (`True`) для каждой строки файла, то для каждой записи выполняется действие `print`, которое просто выводит эту строку на экран.

Блок `{ print }` выполняется по умолчанию, поэтому его можно и не использовать для вывода всех строк файла:



Также можно и не определять условие `1` (`True`), тогда в таком случае, нужно вписать в команду блок `{ print }` :

```
1 awk '{ print }' file.txt
```

Если вместо `1` подставить `0` (`False`), что всегда является ложным, **AWK** не будет выполнять блок кода. В результате ничего не будет выведено на экран:

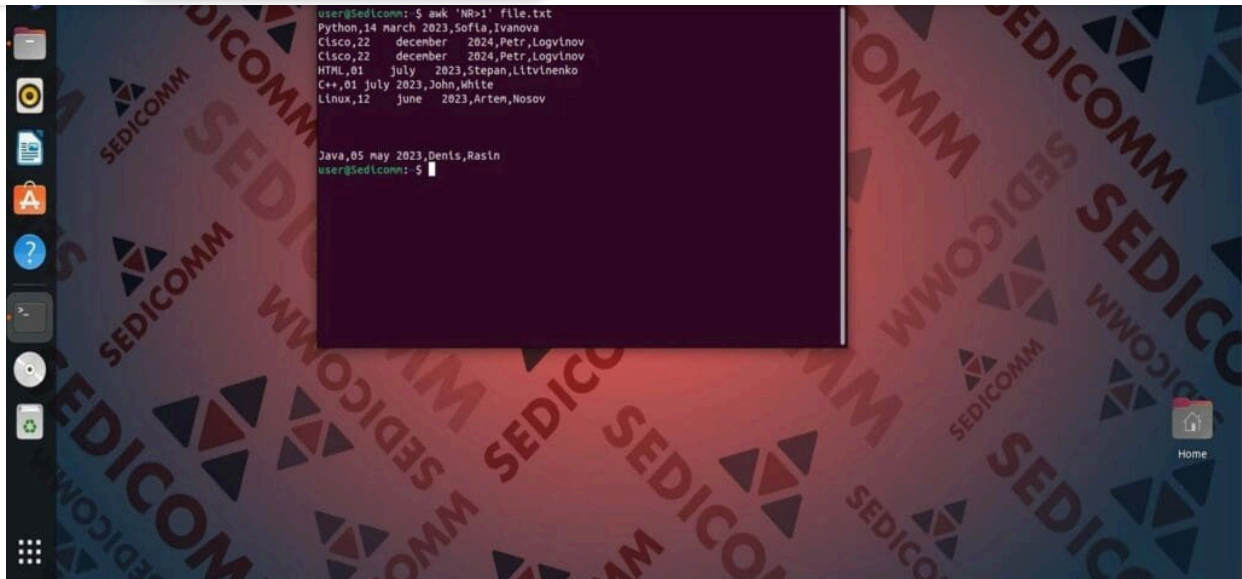
```
1 awk 0 file.txt
```

Вывод определенного диапазона строк

Чтобы ввести на экран строки из нужного Вам **диапазона**, используйте переменную `NR`, которая содержит номер текущей обрабатываемой строки.

Чтобы отобразить на экране все строки файла, кроме первой – выполните условие `NR>1`. Оно указывает, что выводить нужно только те строки, порядковый номер которых больше 1:

```
1 awk 'NR>1' file.txt
```

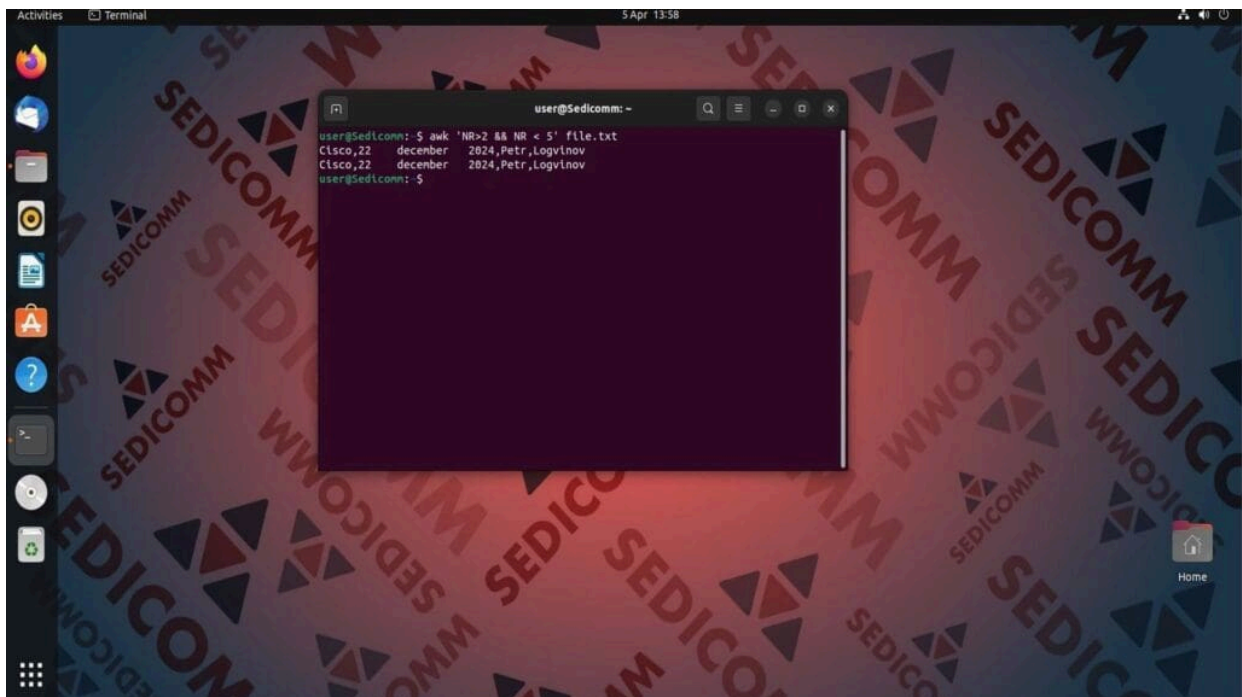


```
user@sedicomm:~$ awk 'NR>1' file.txt
Python,14 march 2023,Sofia,Ivanova
Cisco,22 december 2024,Petr,Logvinov
Cisco,22 december 2024,Petr,Logvinov
HTML,01 july 2023,Stepan,Litvinenko
C++,01 july 2023,John,White
Linux,12 june 2023,Arten,Nosov

user@sedicomm:~$
```

Чтобы вывести на экран текст, который находится между двумя определенными строками, выполните следующий код в терминале:

```
1 awk 'NR>num1 && NR < num2' file.txt
```



```
user@sedicomm:~$ awk 'NR>2 && NR < 5' file.txt
Cisco,22 december 2024,Petr,Logvinov
Cisco,22 december 2024,Petr,Logvinov

user@sedicomm:~$
```

Показанный на скриншоте код возвращает строки из файла, которые находятся между второй и пятой строками, не включая их:

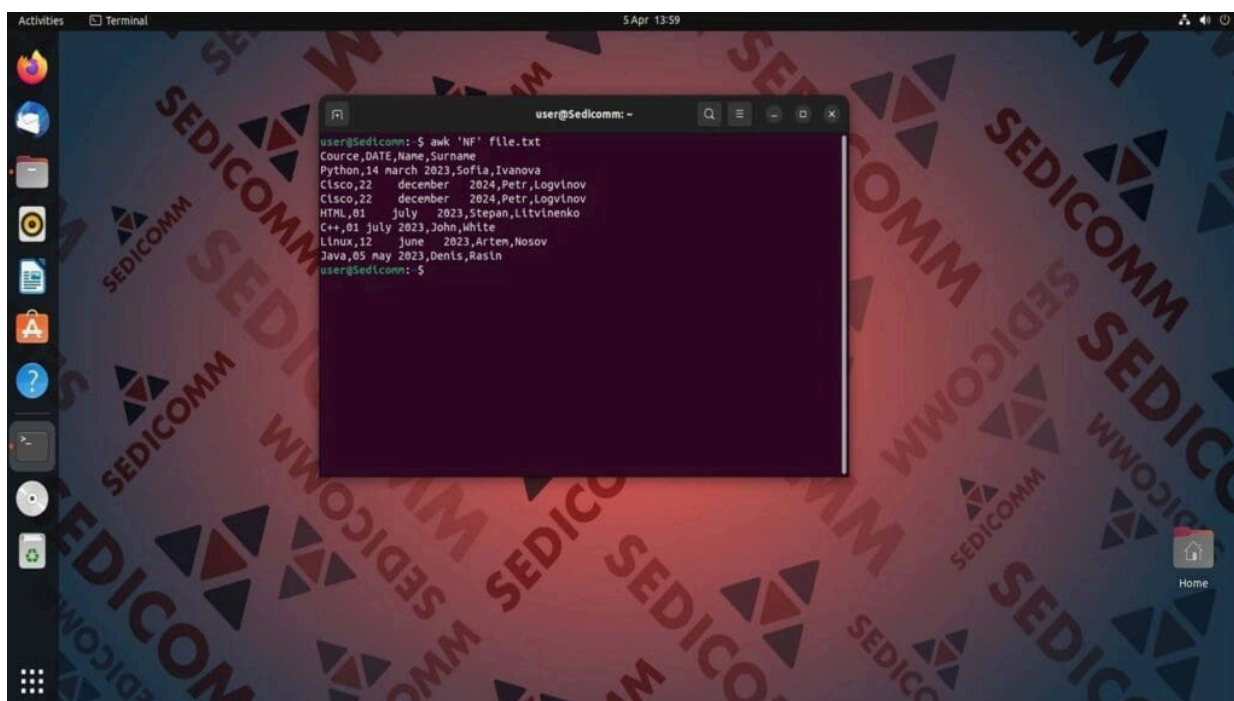
- `&&` – логический оператор «И»;
- `NR<5` – условие, при котором **AWK** выбирает строки, для которых номер строки строго меньше 5.

Таким образом, команда выводит на экран строки с номерами 3 и 4.

Вывод всех строк, которые содержат хотя бы одно поле

Как было сказано ранее, наша таблица не отформатирована и содержит в себе пустые строки. Допустим, нам нужно отобразить только те строки, которые имеют хоть какую-то информацию. Чтобы это сделать, выполним команду:

```
1 awk 'NF' file.txt
```



```
user@SedComm: ~  
user@SedComm:~$ awk 'NF' file.txt  
Source,DATE,Name,Surname  
Python,14 march 2023,Sofia,Ivanova  
Cisco,22 december 2024,Petr,Logvinov  
Cisco,22 december 2024,Petr,Logvinov  
HTML,01 july 2023,Stepan,Litvinenko  
C++,01 july 2023,John,White  
Linux,12 june 2023,Arten,Nosov  
Java,05 may 2023,Denis,Rasin  
user@SedComm:~$
```

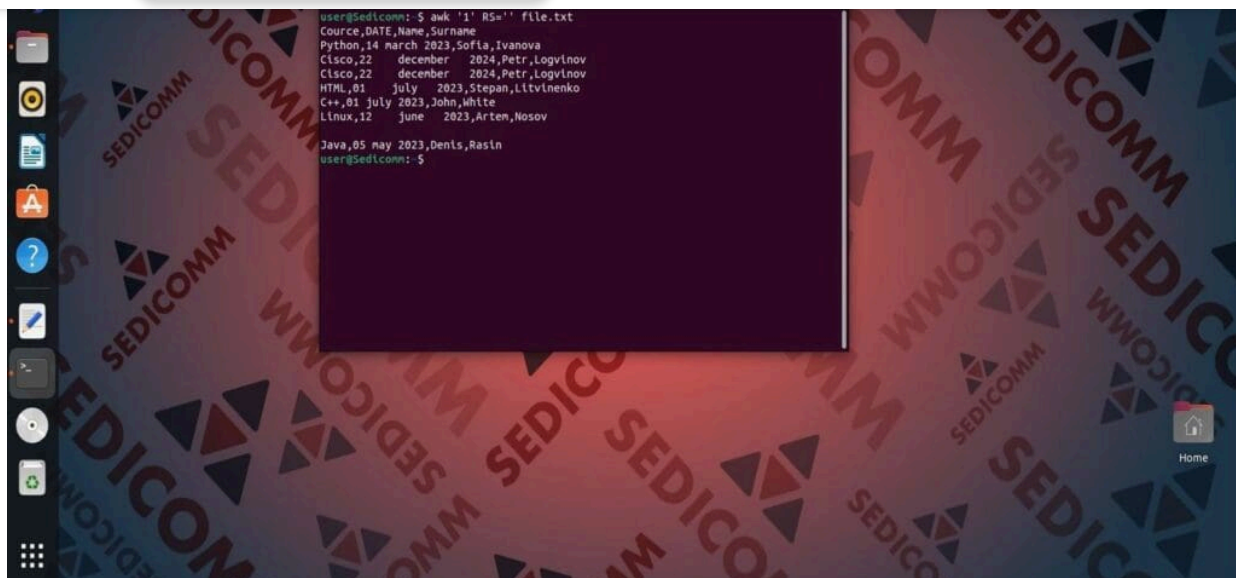


то условие возвращает `1`, и строка выводится. Таким образом, указанная команда позволяет вывести все строки файла, которые содержат хотя бы одно заполненное поле.

Вывод всех строк, которые содержат хотя бы одно поле или только пробелы

В предыдущем примере, мы использовали код **AWK** для проверки наличия непустой строки (содержащей символы, отличные от пробелов и табуляций). Однако, в нашем файле есть та, которая содержит только пробелы. Чтобы вывести на экран ее вместе с остальными заполненными строками, введем в командной строке:

```
1 awk '1' RS=' ' file.txt
```



Извлечение столбцов из файла

Одним из основных преимуществ является возможность извлечения и обработки конкретных столбцов из файла. Для этой задачи используется переменная `$n`, где `n` — это номер столбца, который нужно извлечь. Например, `$1`:

```
1 awk '{ print $1}' file.txt
```

Вывод на экран нескольких столбцов

Если необходимо извлечь сразу несколько столбцов из файла, их номера должны быть перечислены через запятую:

```
1 awk '{ print $1, $3}' FS=, OFS=, file.txt
```



```
user@sedicomm: ~$ awk '{ print $1, $3 }' FS=, OFS=, file.txt
Course,Name
Python,Sofia
Cisco,Petr
HTML,Stepan
C++,John
Linux,Arten
Java,Denis
user@sedicomm: ~$
```

В этом примере:

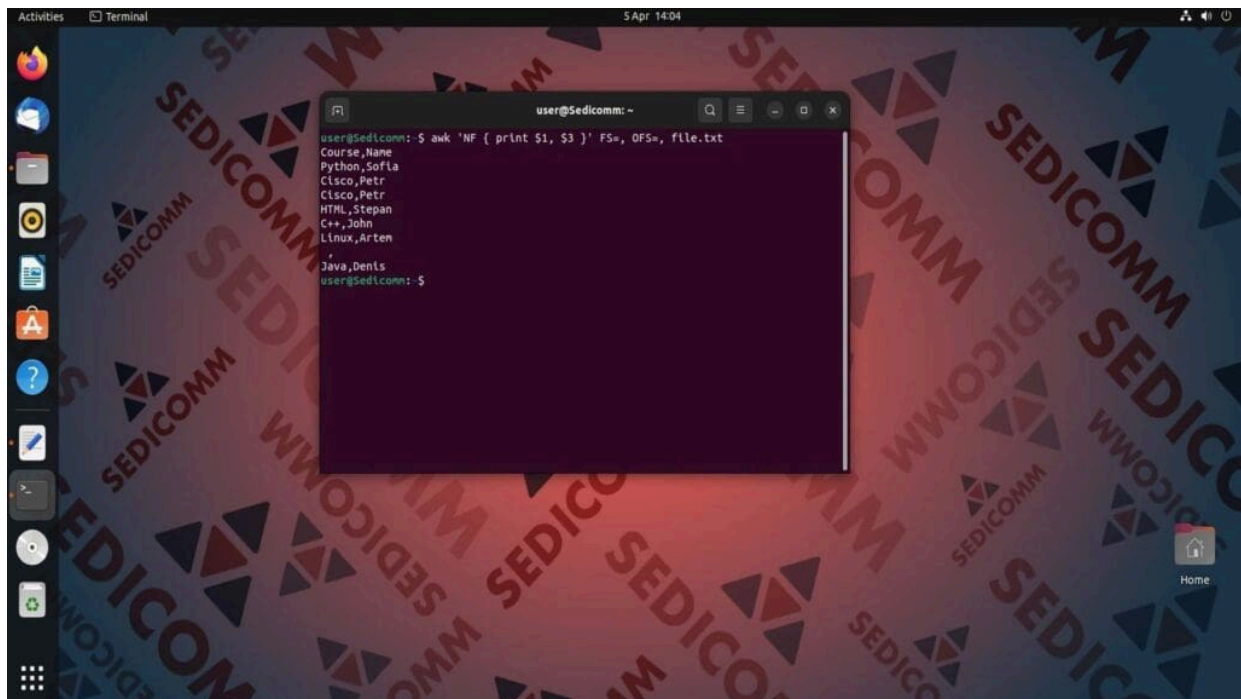
- `FS=,` – устанавливает разделителем полей запятую при считывании данных из файла;
- `OFS=,` – устанавливает запятую как разделитель выведенных столбцов на экране;
- `{ print $1, $3 }` – указывает на то, что нужно вывести первое и третье поле из каждой строки файла.

Таким образом, после выполнения этой команды, в выводе останутся только первый и третий столбец исходного файла, разделенные запятой.

Важно: при использовании этого метода пустые строки также будут разделены запятыми.

Чтобы вывести на экран только непустые строки (также те, которые состоят только из пробелов), используйте команду:

--	--



Вывод на экран нескольких столбцов с помощью BEGIN

Вы также можете использовать блок `BEGIN` в команде для вывода нескольких столбцов файла. Модифицируем код из предыдущего примера следующим образом:

```
1 awk 'BEGIN { FS=OFS="," } NF { print $1, $3 }' file
```



```
user@sedicomm:~$ awk 'BEGIN { FS=OFS="," } NF { print $1, $3 }' file.txt
Course,Name
Python,Sofia
Cisco,Petr
Cisco,Petr
HTML,Stepan
C++,John
Linux,Arten
Java,Denis
user@sedicomm:~$
```

Ключевое слово **BEGIN** позволяет задать начальные условия для выполнения программы **AWK** перед тем, как начнется обработка входного файла. В данном случае, оно устанавливает значения для **FS** и **OFS**.

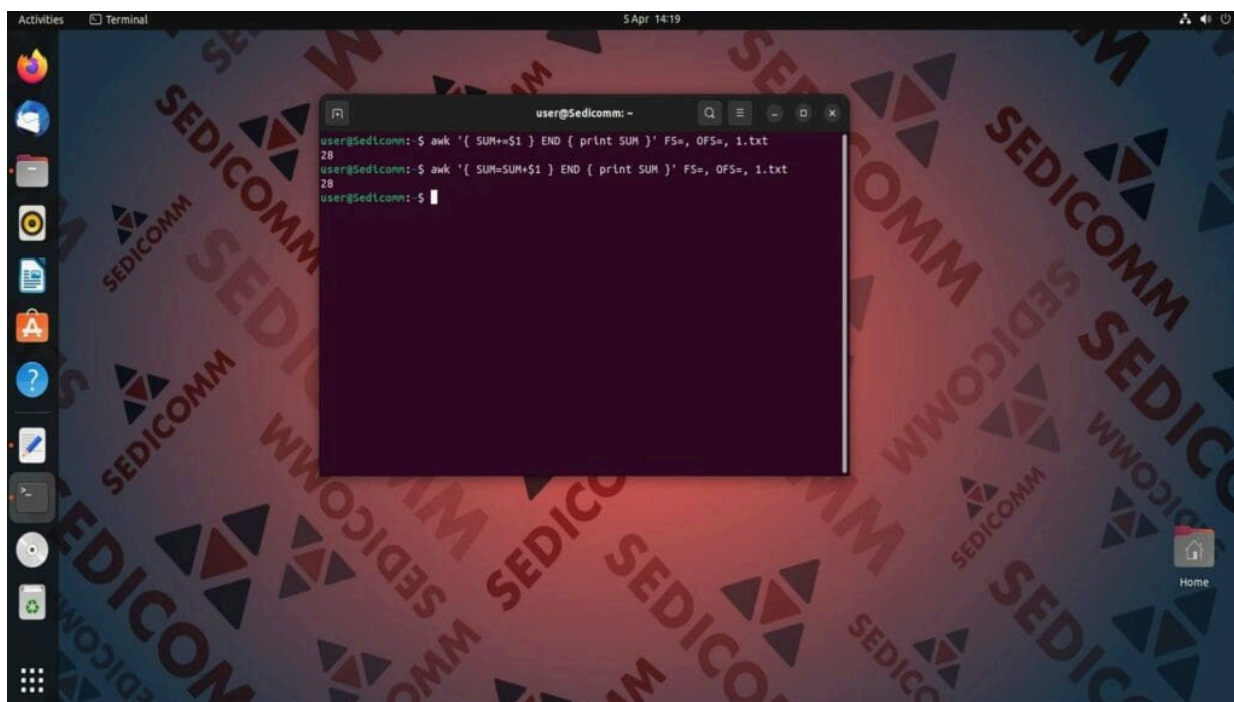
Математические операции в столбцах

Скриптовый язык **AWK** позволяет пользователям выполнять простые арифметические операции. Например, чтобы подсчитать сумму чисел первого столбца файла **1.txt**, выполним одну из представленных команд:

```
1 awk '{ SUM=SUM+$1 } END { print SUM }' FS=, OFS=, 1.txt
```

Или:

--	--



```
user@Sedcomm: ~  
user@Sedcomm:~$ awk '{ SUM+= $1 } END { print SUM }' FS=, OFS=, 1.txt  
28  
user@Sedcomm:~$ awk '{ SUM+= $1 } END { print SUM }' FS=, OFS=, 1.txt  
28  
user@Sedcomm:~$
```

При решении этой задачи нам потребовалась собственная переменная для хранения промежуточных значений `SUM` . Детальнее разберем конструкцию команды:

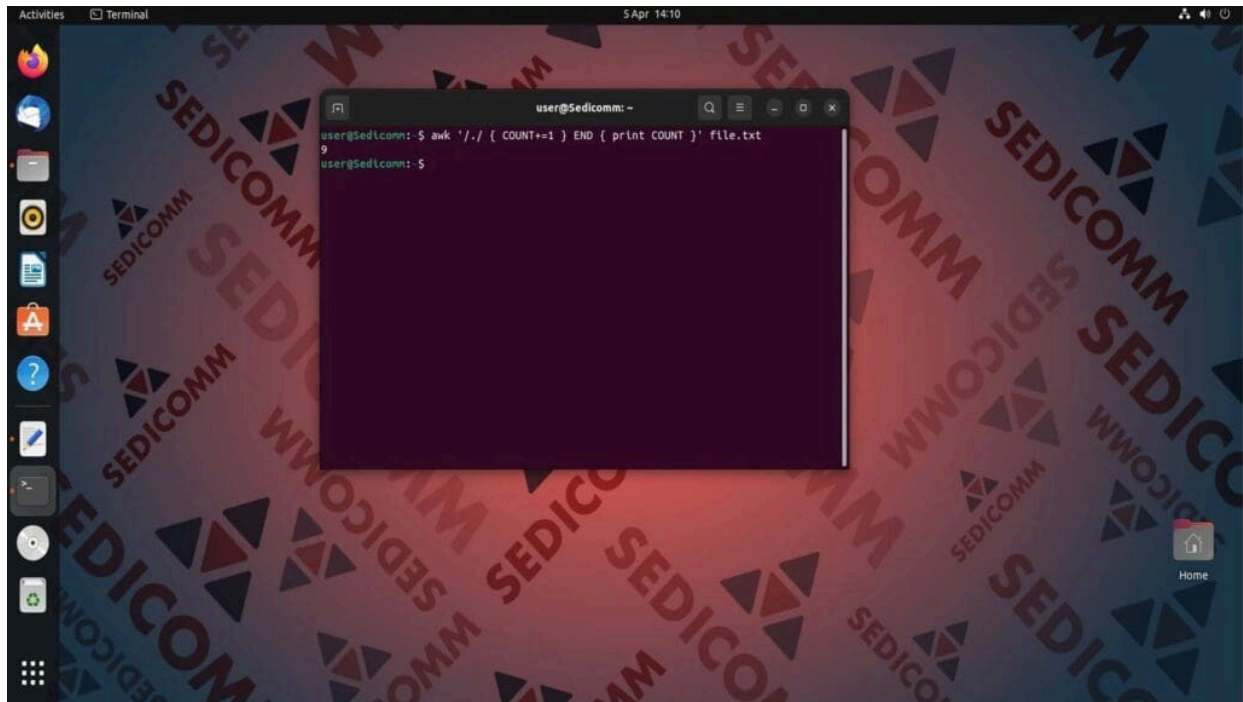
- `FS=, OFS=,` – задает разделитель полей для входного и выходного файла как запятую;
- `{ SUM=SUM+$1 }` – поочередно перебирает каждую строку файла и прибавляет значение первого столбца к переменной `SUM` ;
- `END { print SUM }` – печатает значение переменной `SUM` .

Таким образом, код вычисляет сумму первого столбца в файле и печатает ее на экран.

Подсчет количества непустых строк

ключевым словом **END** :

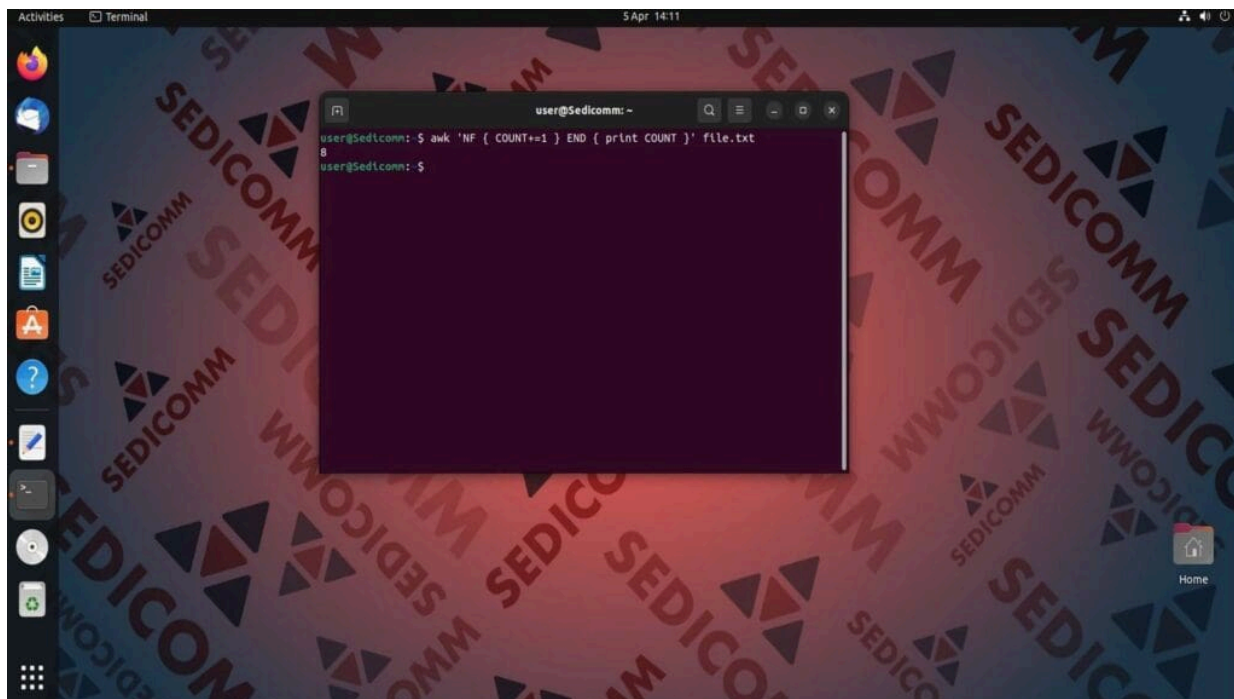
```
1 awk '/./ { COUNT+=1 } END { print COUNT }' file.txt
```



В этой конструкции содержатся несколько элементов, которые нужно объяснить:

- `/./` – шаблон поиска, который соответствует любой строке, содержащей хотя бы один непробельный символ;
- `{ COUNT+=1 }` – выполняется для каждой строки и увеличивает значение переменной `COUNT` на единицу;
- `END { print COUNT }` – после обработки всех строк выводит значение переменной `COUNT` .

Чтобы вывести на экран число строк, которые содержат реальные данные (не пробелы или табуляции), введите в командной строке:

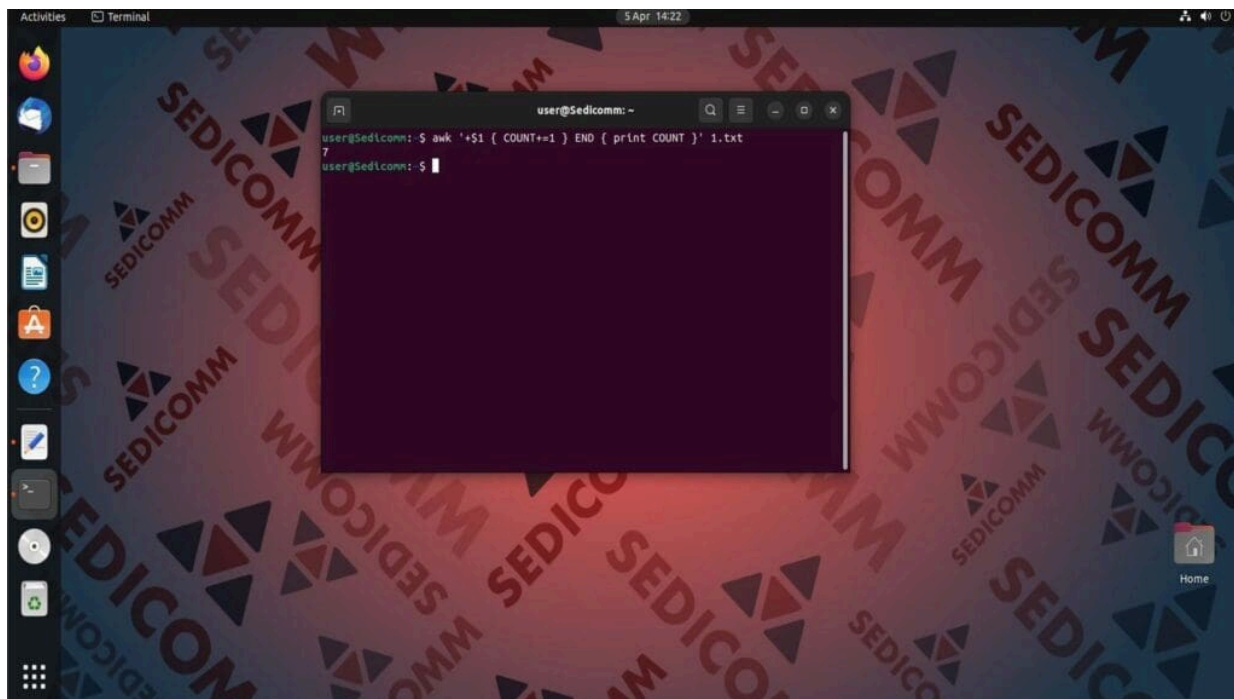


```
user@Sedcomm: ~  
user@Sedcomm:~$ awk 'NF { COUNT++ } END { print COUNT }' file.txt  
8  
user@Sedcomm:~$
```

Здесь используется встроенная переменная `NF`, которая представляет количество полей в каждой строке файла. Если значение `NF` для строки не равно нулю (т.е. строка не пуста), то переменная `COUNT` увеличивается на 1. В конце обработки файла с помощью блока `END` скриптовый язык **AWK** выводит значение переменной `COUNT`, которая представляет собой количество строк, имеющих один или более символов.

Подсчет количества полей, которые содержат только числа

Поговорим о том, как определить какие данные содержат поля строки. Мы знаем, что в файле `1.txt` первое поле каждой строки, кроме первой, имеет числовое значение. С помощью языка сценариев мы можем подсчитать их количество, выполнив команду:



Подробнее разберем код:

1. `+$1` – проверяет, является ли значение первого поля в строке не равным нулю (также мы используем унарный плюс, чтобы преобразовать строку в число).
2. `{ COUNT+=1 }` – увеличивает переменную `COUNT` на единицу, если условие истинно.
3. `END { print COUNT }` – выводит значение переменной `COUNT` на экран после обработки всего файла.

Использование массивов в AWK

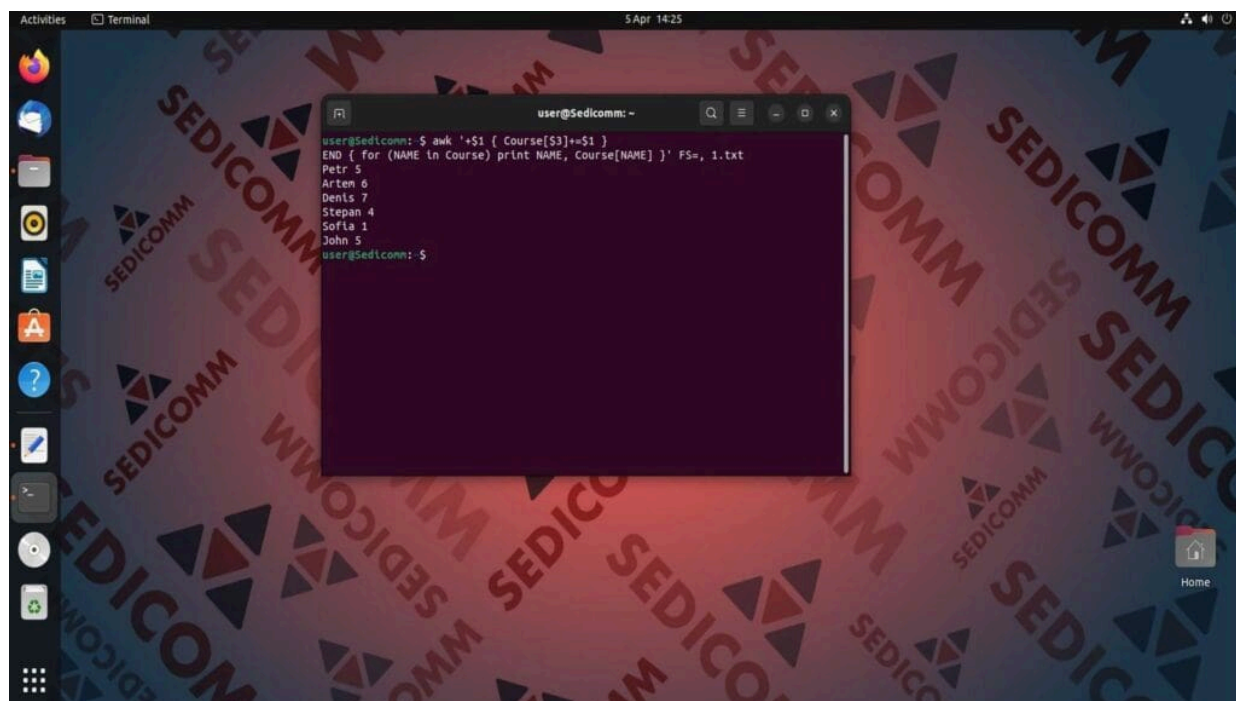
Язык **AWK** имеет встроенную поддержку **массивов**, что делает его мощным инструментом для анализа структурированных данных. Массивы в **AWK** – ассоциативные, то есть они используются для хранения пар «ключ-

используется имя массива и ключ в квадратных скобках.

Сопоставление данных из двух столбцов

Рассмотри на примере. В файле `1.txt` в первом столбце хранятся числа, а в третьем имена людей. Допустим, нам нужно вывести на экран два столбца (первый и третий). Однако, одно из имен (`Petr`) повторяется в таблице, то есть ему соответствует сразу два числа. В таком случае, при сопоставлении эти два числа суммируются:

```
1 awk '+$1 { Course[$3]+=$1 }  
2 END { for (NAME in Course) print NAME, Course[NAME] }' FS=, 1.txt
```



```
user@Sedcomm: ~  
$ awk '+$1 { Course[$3]+=$1 }  
END { for (NAME in Course) print NAME, Course[NAME] }' FS=, 1.txt  
Petr 11  
Arten 6  
Denis 7  
Stepan 4  
Sofia 1  
John 5  
user@Sedcomm: ~$
```

Действия, которые выполнила команда:



который использует третий столбец (имена) в качестве индекса.

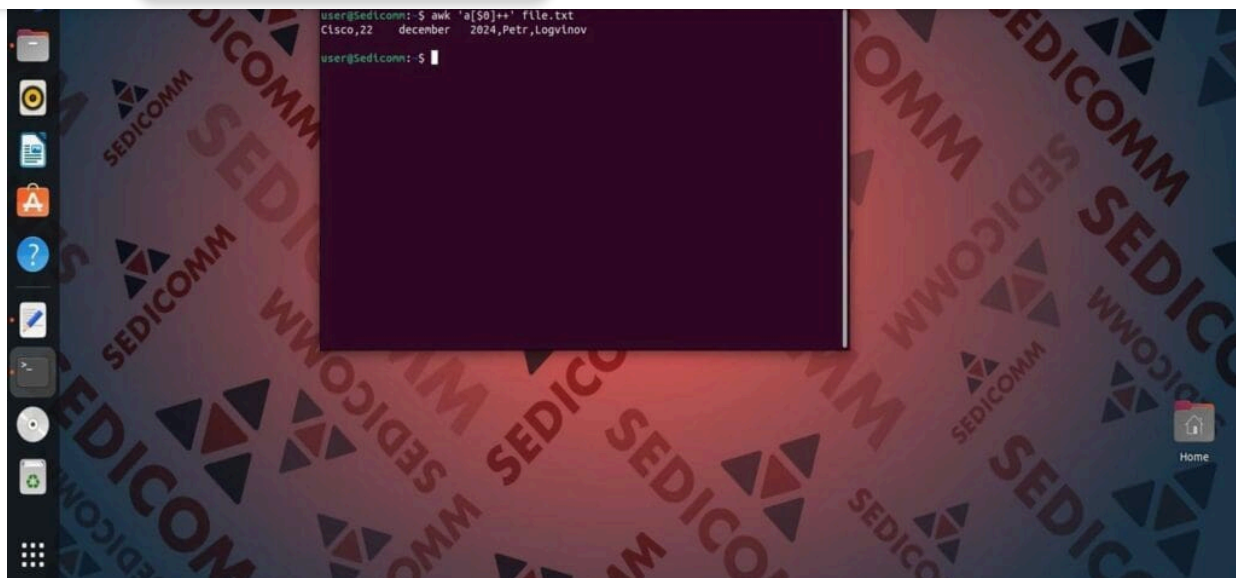
2. Блок `END` обрабатывает все элементы массива `Course` и выводит их на экран, используя цикл `for`. Для каждого элемента функция `print` выводит соответствующее имя (индекс массива) и сумму чисел.
3. Опция `FS=,` задает запятую в качестве разделителя полей.

Важно: если в файле содержатся строки без положительных чисел в первом столбце, они будут проигнорированы.

Определение повторяющихся строк с использованием AWK

В таблице могут содержаться повторяющиеся строки. И в большом файле довольно сложно отыскать их самостоятельно. Однако, использование массивов поможет легко автоматизировать этот процесс:

```
1 awk 'a[$0]++' file.txt
```

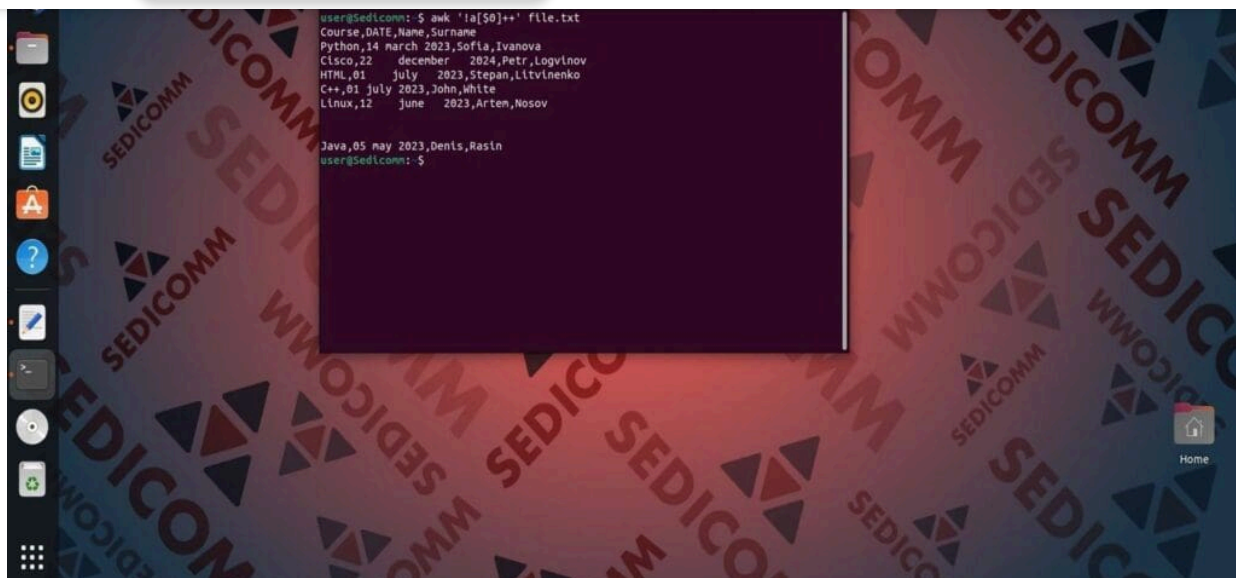


Этот код отображает только уникальные строки в файле. Когда **AWK** читает строки, они сохраняются в массиве `a`. Если **AWK** встречает строку, которая уже есть в массиве `a`, он ее игнорирует. Если же встречает новую строку – добавляет ее в массив `a` с помощью оператора `++`. В результате на экран выводятся строки, которые встретились более одного раза.

Игнорирование повторяющихся строк

Данная задача противоположна предыдущей. Однако, в данном случае, выводятся строки без повторов. Если строка имеет дубликат – он игнорируется, а оригинальная запись записывается в стандартном выводе. Например:

```
1 awk '!a[$0]++' file.txt
```



```
user@sedicomm: ~$ awk '1a{$0}++' file.txt
Course,DATE,Name,Surname
Python,14 march 2023,Sofia,Ivanova
Cisco,22 december 2024,Petr,Logvinov
HTML,01 july 2023,Stepan,Litvinenko
C++,01 july 2023,John,White
Linux,12 june 2023,Arsen,Nosov

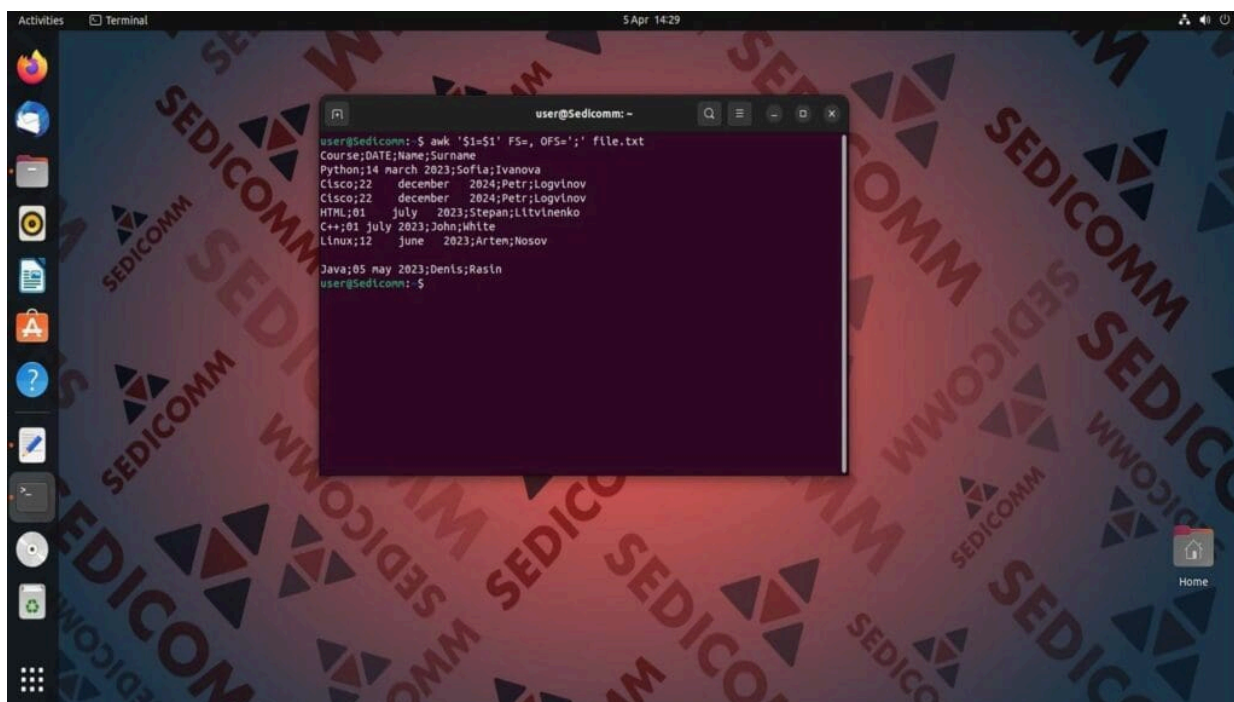
user@sedicomm: ~$
```

Использование временных переменных FS и OFS

В AWK встроены две переменные: `FS` и `OFS`, которые используются для указания разделителя полей между столбцами во входных и выходных данных, соответственно. Скриптовый язык позволяет задавать им собственные значения, чтобы изменить вид содержимого при выводе на экран.

Изменение разделителей полей

Как Вы знаете, все поля в нашей исходной таблице разделены пробелами. Допустим, мы хотим заменить пробелы на точку запятой. Для этого будем использовать следующую команду:



```
user@Sedicom: ~  
user@Sedicom: $ awk 'FS=, OFS=;' file.txt  
Course;DATE;Name;Surname  
Python;14 march 2023;Sofia;Ivanova  
Cisco;22 december 2024;Petr;Logvinov  
Cisco;22 december 2024;Petr;Logvinov  
HTML;01 july 2023;Stepan;Litvinenko  
C++;01 july 2023;John;White  
Linux;12 june 2023;Arten;Nosov  
  
Java;05 may 2023;Denis;Rasin  
user@Sedicom: $
```

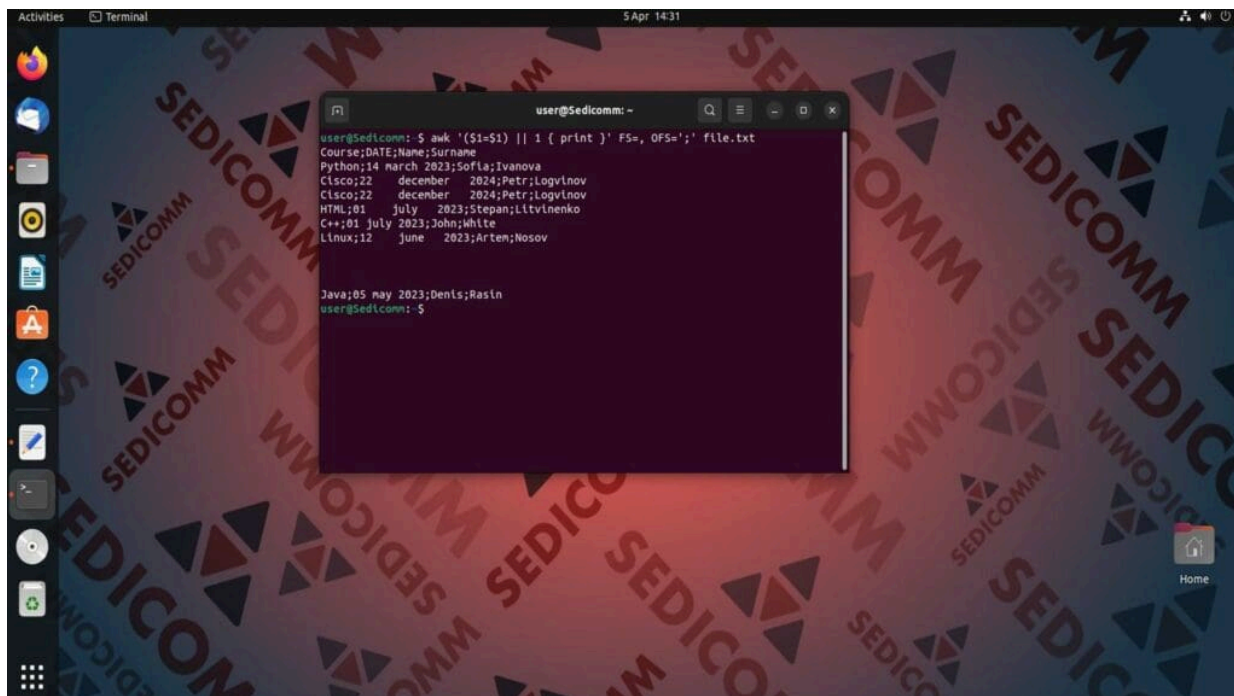
Здесь:

1. `FS=,` – задает символ запятой в качестве разделителя полей входного файла.
2. `OFS=';'` – задает символ точки с запятой в качестве разделителя полей выходного файла.
3. Операция `$1=$1` присваивает первому полю в строке его же значение, таким образом происходит переформатирование строки. Как результат, пробелы и табуляции между полями удаляются.

В результате, каждая строка будет выведена с новым разделителем полей, заданным переменной `OFS`. То есть, данная команда просто изменяет разделитель полей из запятой в точку с запятой.

Вы могли заметить, что при выполнении команды пустые строки были проигнорированы. Если Вы все-таки хотите оставить их в выводе,

```
1 awk '($1=$1) || 1 { print }' FS=, OFS=';' file.txt
```



```
user@Sedcomm: ~  
user@Sedcomm: $ awk '($1=$1) || 1 { print }' FS=, OFS=';' file.txt  
Course;DATE;Name;Surname  
Python;14 march 2023;Sofia;Ivanova  
Cisco;22 december 2024;Petr;Logvinov  
HTML;01 july 2023;Stepan;Litvinenko  
C++;01 july 2023;John;White  
Linux;12 june 2023;Artem;Nosov  
Java;05 may 2023;Denis;Rasin  
user@Sedcomm: $
```

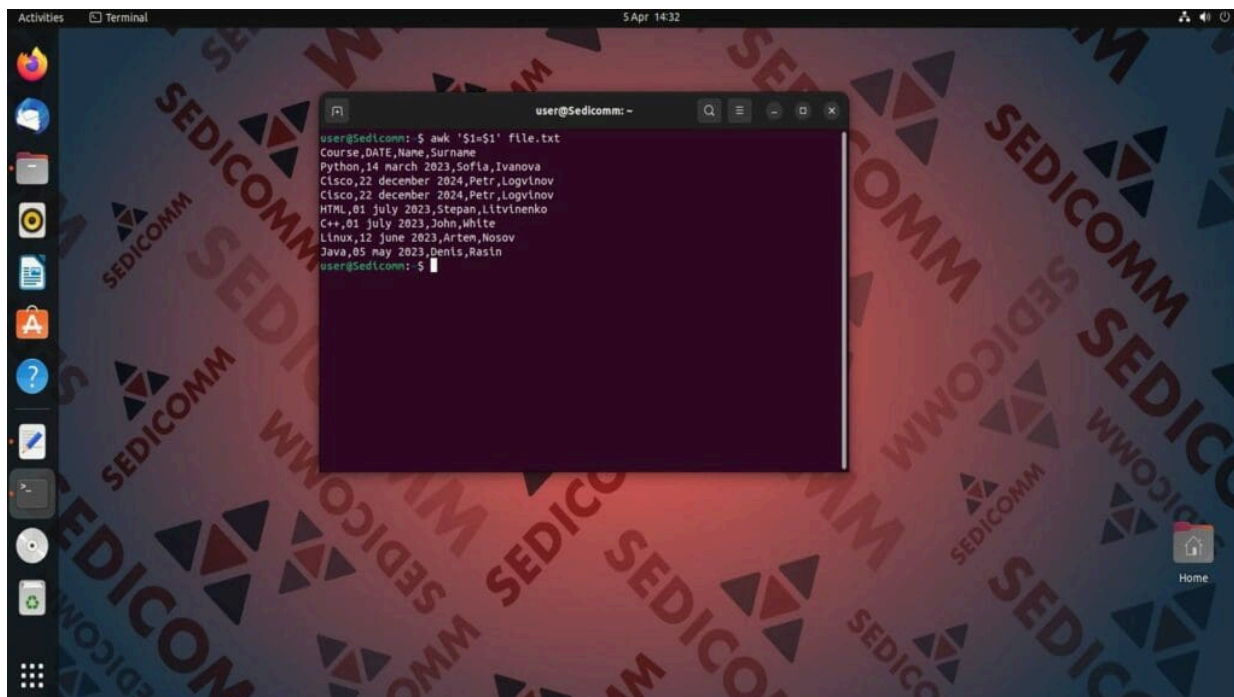
Объясним это выражение:

1. Первый элемент `$1` устанавливается равным себе `$1=$1`, что приводит к удалению ведущих и конечных пробелов.
2. Если первый элемент в строке теперь равен себе, то выражение `$1=$1` возвращает `True`.
3. Если первый элемент в строке уже был пустым или равен строке пробелов, то выражение `$1=$1` возвращает `False`.
4. `|| 1` используется для того, чтобы всегда вернуть `True`, если первый элемент был пустым или равен строке пробелов.

Удаление лишних пробелов

таблицу и разделить слова одинарными пробелами, введите команду в терминале:

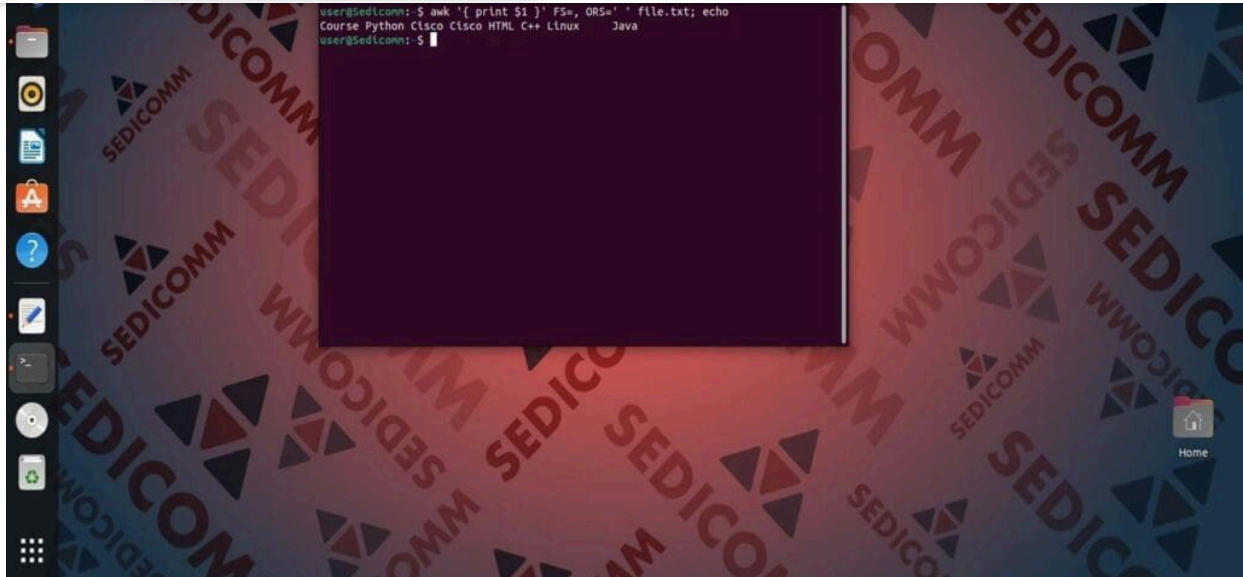
```
1 awk '$1=$1' file.txt
```



Отображение столбцов в линию

Форматирование столбцов – одна из главных функций **AWK**. Благодаря ей Вы сможете изменить вид столбца. Например, вывести колонку в строку. Давайте расположим в строку первый столбец файла:

```
1 awk '{ print $1 }' FS=, ORS=' ' file.txt; echo
```

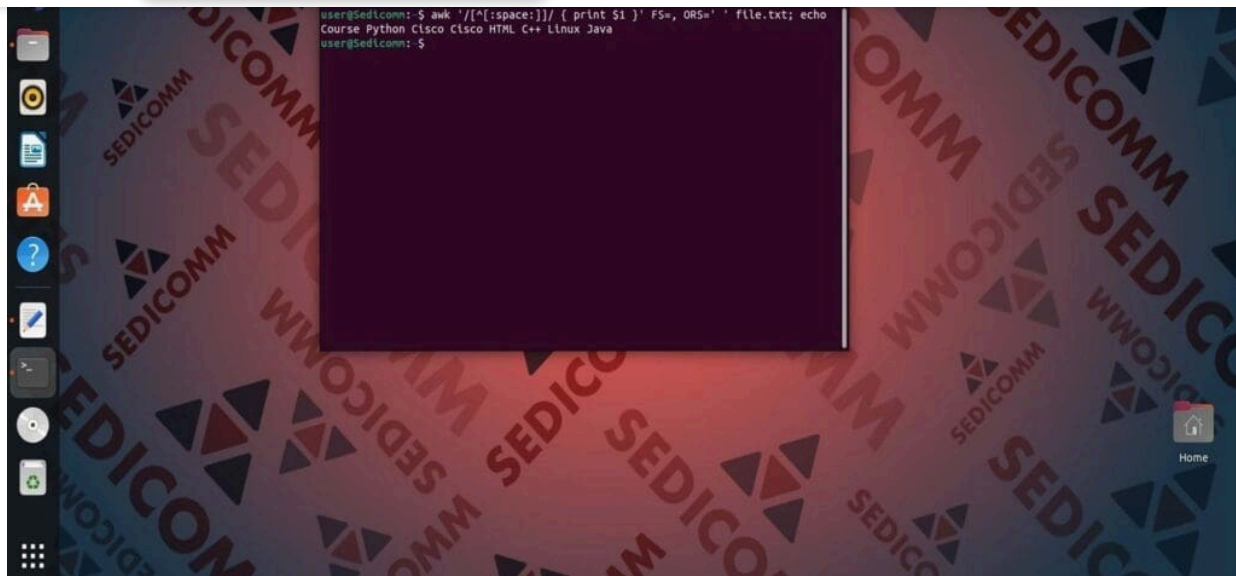


Разберем код подробнее:

- `FS=,` – указывает **AWK** на использование запятой в качестве разделителя полей;
- `ORS=' '` – задает пробел в качестве разделителя строк при выводе результатов;
- `echo` – добавляет в конце символ переноса строки, чтобы разделить вывод **AWK** и приглашение оболочки.

Чтобы удалить из вывода пустые поля, воспользуйтесь этим регулярным выражением:

```
1 awk '/[^[[:space:]]/ { print $1 }' FS=, ORS=' ' file.txt; echo
```



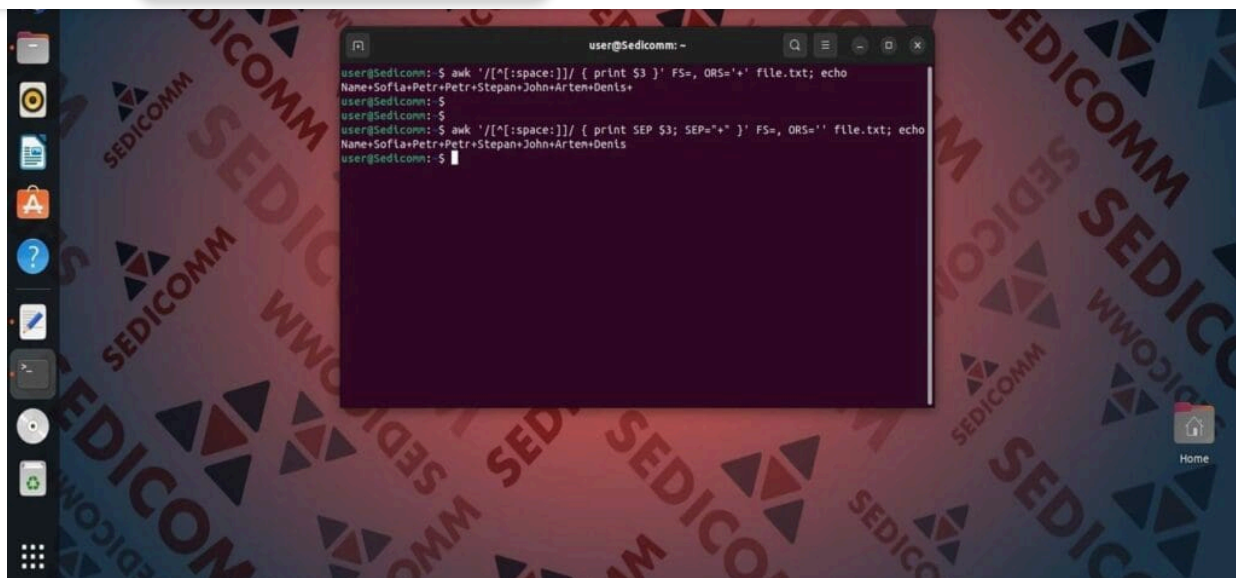
Данный код возвращает первый столбец каждой строки файла, игнорируя пустые строки и те, которые состоят только из пробельных символов. Команда `/^[[:space:]]/` означает, что только строки, содержащие хотя бы один символ, который не является пробелом, будут выбраны для обработки.

Для лучшей читабельности изменим символ разделитель ORS на `+` и выведем таким образом на экран третий столбец:

```
1 awk '/^[[:space:]]/{ print $3 }' FS=, ORS='+' file; echo
```

Однако, тогда в конце строки появляется лишний плюс. Чтобы убрать его, модифицируйте предыдущую конструкцию следующим образом:.

```
1 awk '/^[[:space:]]/{ print SEP $3; SEP="+" }' FS=, ORS='' file.t
```



```
user@Sedcomm: ~  
user@Sedcomm:~$ awk '/[[:space:]]/ { print $3 }' FS=, ORS="+ " file.txt; echo  
Name+Sofia+Petr+Petr+Stepan+John+Artem+Denis+  
user@Sedcomm:~$  
user@Sedcomm:~$ awk '/[[:space:]]/ { print SEP $3; SEP="+" }' FS=, ORS="" file.txt; echo  
Name+Sofia+Petr+Petr+Stepan+John+Artem+Denis  
user@Sedcomm:~$
```

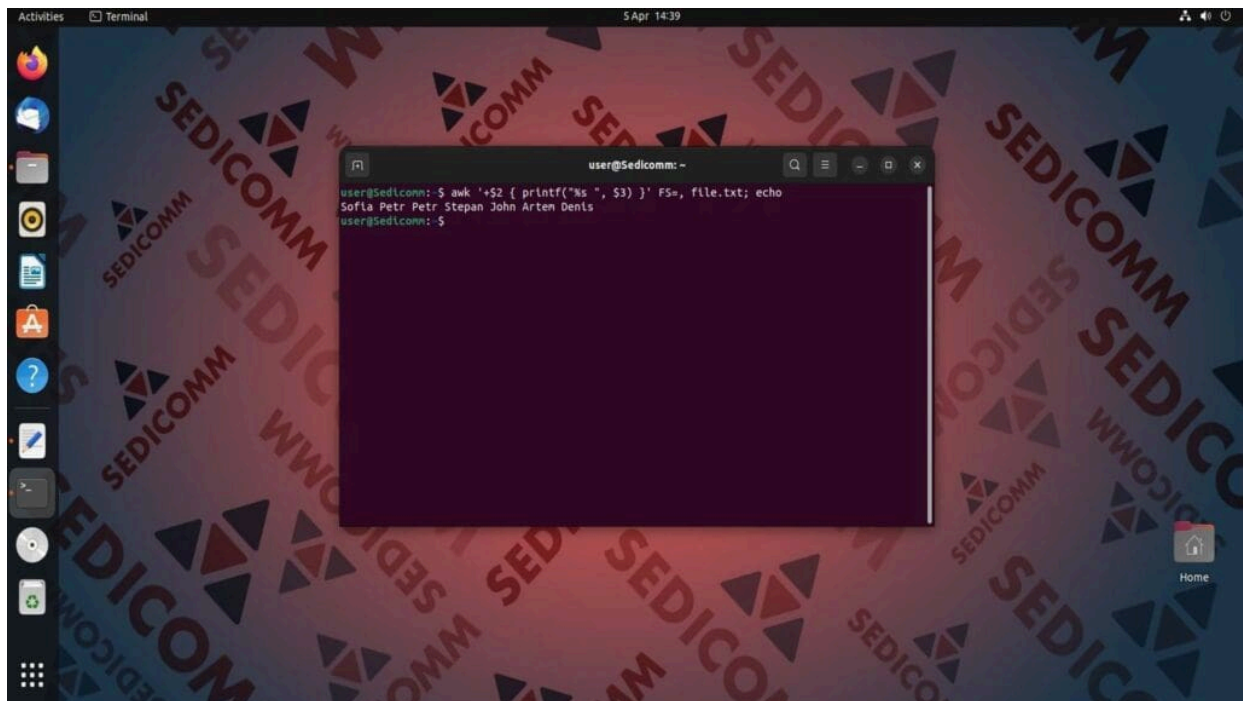
После последнего поля разделитель `+` не нужен, так что используется переменная `SEP`, которая инициализируется пустой строкой. Эта переменная определяет, что поле последнее и не выводит символ разделитель после него.

Форматирование полей

AWK позволяет **форматировать вывод полей** с помощью специальных функций, таких как `printf` и `sprintf`. Первая позволяет задавать формат вывода с использованием специальных символов, таких как `%s` для строчного типа данных, `%d` для целых чисел и т. д. Вторая работает аналогичным образом, но возвращает отформатированную строку в качестве значения, которое затем можно присвоить переменной.

Для примера давайте выведем третью колонку в одну строку:

--	--



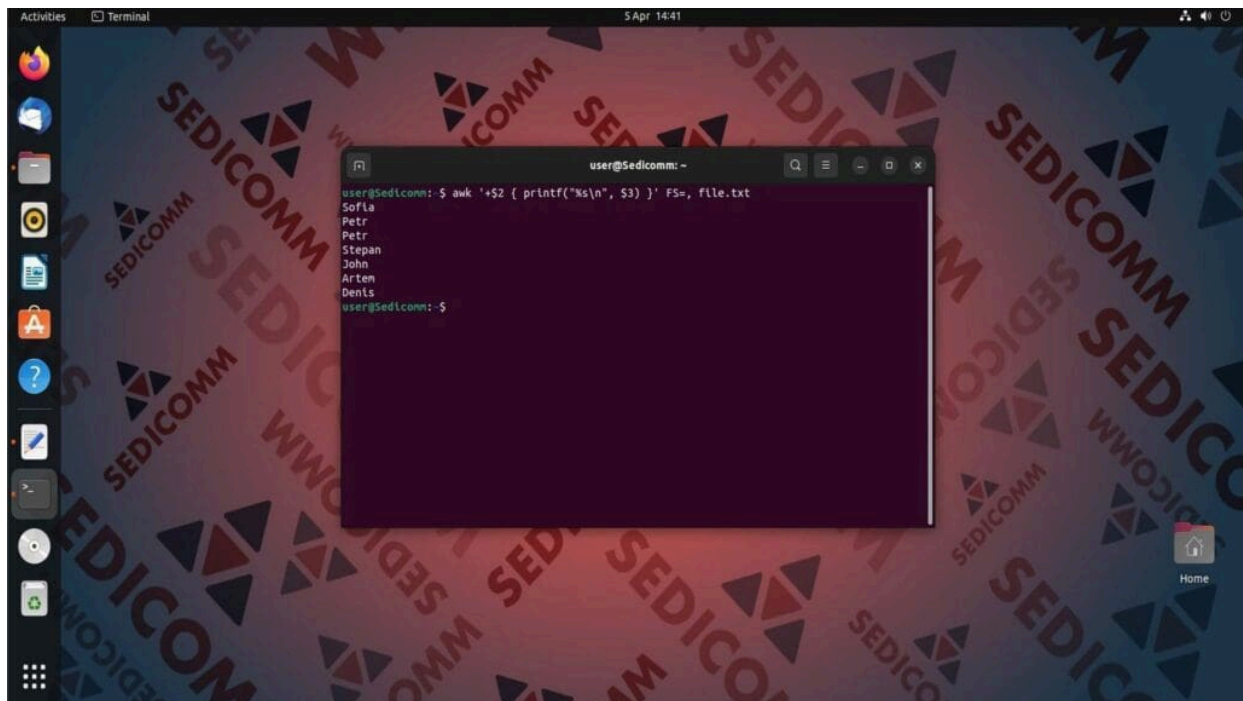
```
user@Sedcomm: ~$ awk '+$2 { printf("%s ", $3) } FS=, file.txt; echo'
Sofia Petr Petr Stepan John Arten Denis
user@Sedcomm: ~$
```

Разберем команду детальнее:

- `%s` – означает, что нужно вывести строку как тип данных;
- `$3` – третье поле текущей строки, считая разделителем запятую (`FS=,`);
- `printf("%s ", $3)` – означает, что нужно вывести текст из третьего поля и добавить пробел после него;
- `+$2` – проверяет, является ли второе поле данными строчного типа;
- `echo` – добавляет символ переноса строки в конце.

Используя специальный символ `\n`, можно расположить полученные значения в столбик:

--	--



Преобразование вывода в таблицу

Для создания таблицы с помощью **AWK** необходимо использовать функции форматирования вывода. Одной из таких функций является `printf`, которая позволяет задавать формат для выводимых данных.

Например, чтобы вывести два столбца (`$3` и `$1`) в виде таблицы с вертикальной чертой (`|`) в качестве разделителя – будем использовать эту команду:

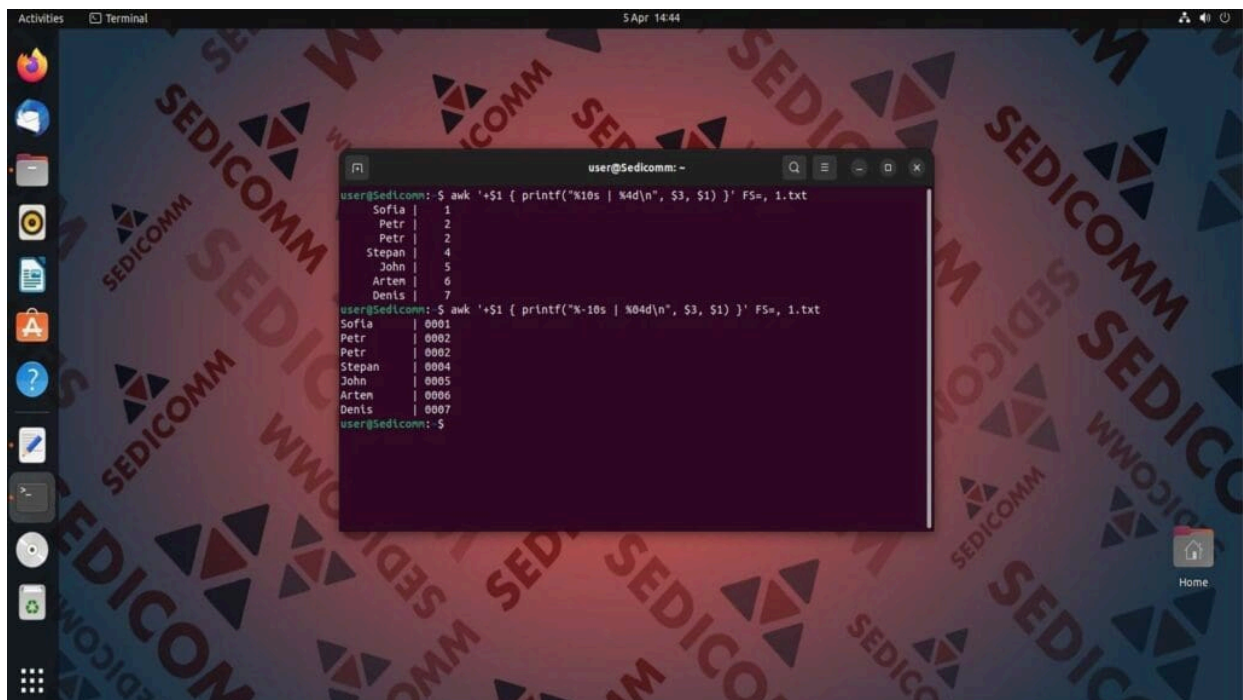
```
1 awk '+$1 { printf("%10s | %4d\n", $3, $1) }' FS=, 1.txt
```

Здесь:

- `%4d` – второй столбец будет шириной в 4 символа и выровнен по правому краю;
- `|` – разделитель между столбцами.

Можно модифицировать вывод таблицы, так чтобы левый столбец был выровнен по левому краю, а пустые поля правого заполнялись нулями вместо пробелов:

```
1 awk '+$1 { printf("%-10s | %04d\n", $3, $1) }' FS=, 1.txt
```



В этой конструкции:

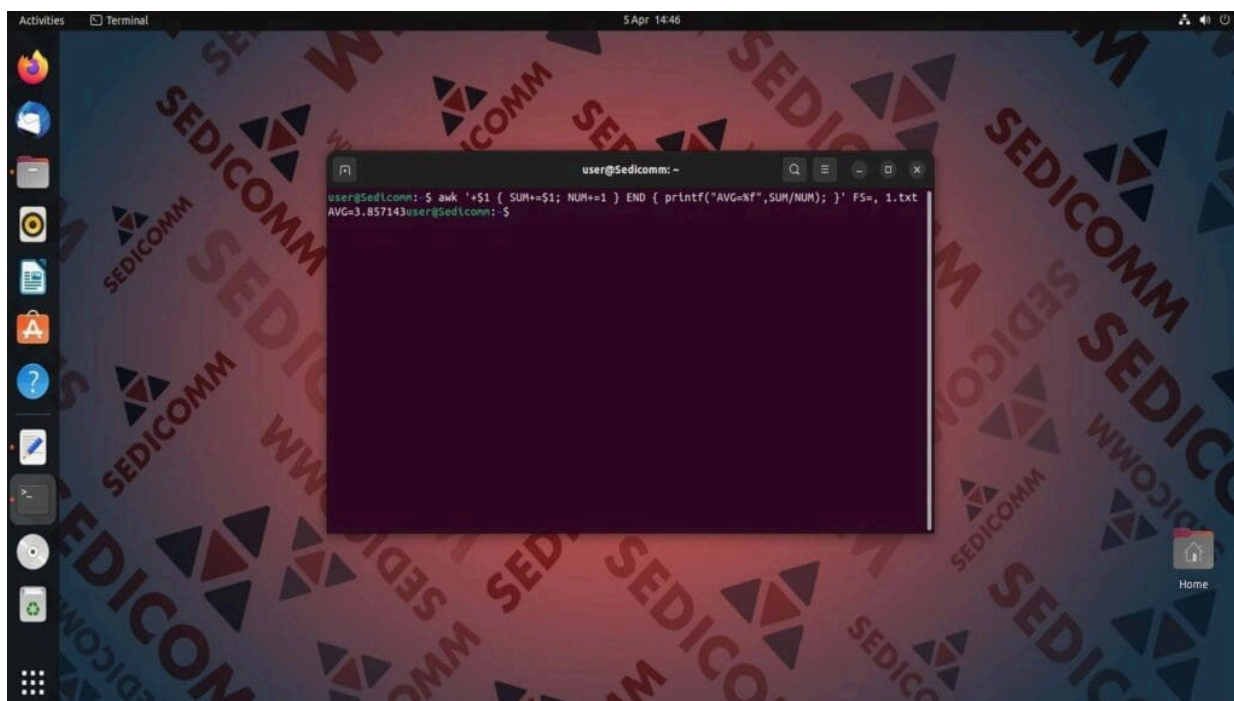
1. `%-10s` – спецификатор формата для строчного типа данных, который указывает, что мы хотим выравнивать запись по левому краю и оставить максимум 10 символов для этого поля. Знак `-` означает, что выравнивание будет выполнено по левому краю.

он добавит соответствующее количество ведущих нулей, если число состоит из менее чем 4 символов.

Операции с числами с плавающей запятой

В AWK для операций с числами с плавающей запятой используется форматирование полей `%f`. С его помощью можно указать количество знаков после запятой и выравнивание значений. К примеру:

```
1 awk '+$1 { SUM+= $1; NUM+=1 } END { printf("AVG=%f",SUM/NUM); }' F
```



Данный код использует скриптовый язык для вычисления среднего значения с запятой в качестве разделителя:



- `{ SUM+=$1; NUM+=1 }` – для каждой строки, где выполняется условие, **AWK** добавляет значение из первого столбца к переменной `SUM` и увеличивает `NUM` на 1;
- `END { printf("AVG=%f",SUM/NUM); }` – после обработки всех строк, язык сценариев делит `SUM` на `NUM`, и выводит результат в отформатированном виде с помощью функции `printf`.

Таким образом, данный код выводит среднее значение всех чисел в первом столбце файла в формате `AVG=значение`.

Использование строчных функций в AWK

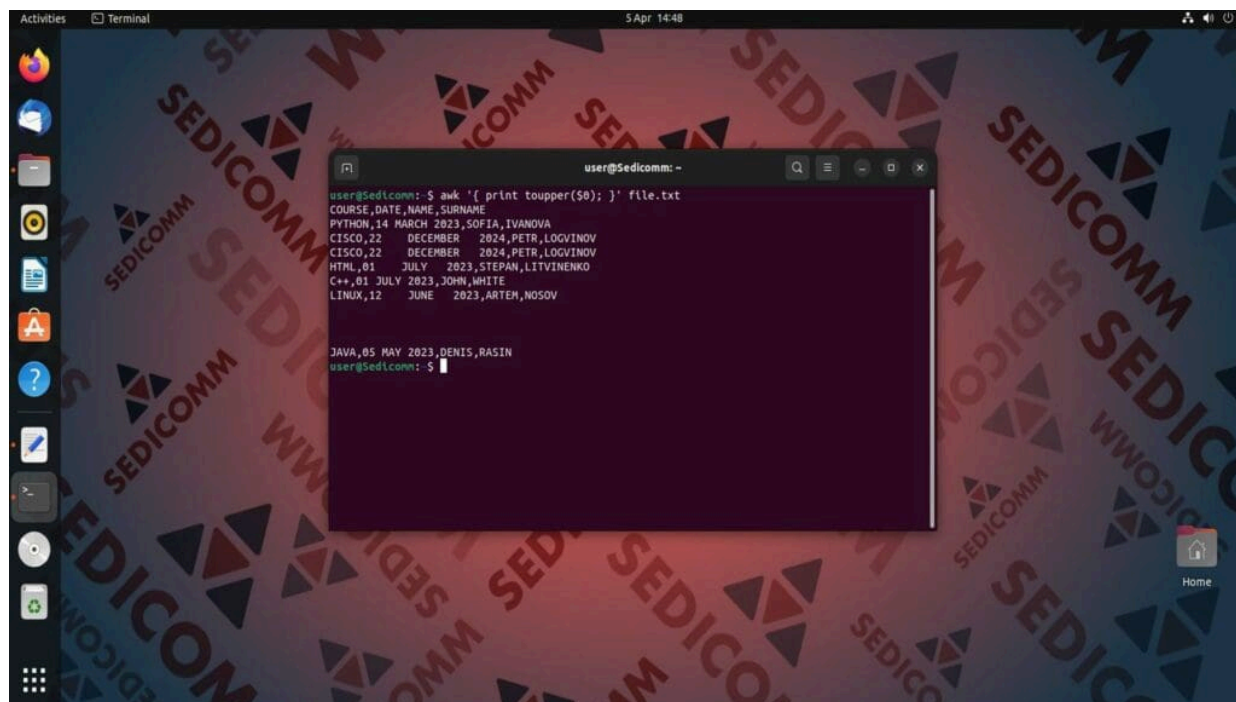
Язык сценариев **AWK** предоставляет ряд встроенных **строчных функций**, которые позволяют выполнять операции с текстом. Некоторые из них:

- `tolower(str)` – преобразование строки в нижний регистр;
- `toupper(str)` – преобразование строки в верхний регистр;
- `length(str)` – возвращение длины строки;
- `index(str, sub_str)` – возвращение позиции первого вхождения подстроки `sub_str` в строке `str`;
- `substr(str, start, length)` – возвращение подстроки строки `str`, начинающуюся с позиции `start` и имеющую длину `length`;

Преобразование строки в верхний регистр

В файле `file.txt` все текстовые поля записаны с маленькой буквы. Допустим, нам нужно перевести все буквы нижнего регистра в верхний:

```
1 awk '{ print toupper($0); }' file.txt
```



Давайте проанализируем элементы выражения

```
{ print toupper($0); } :
```

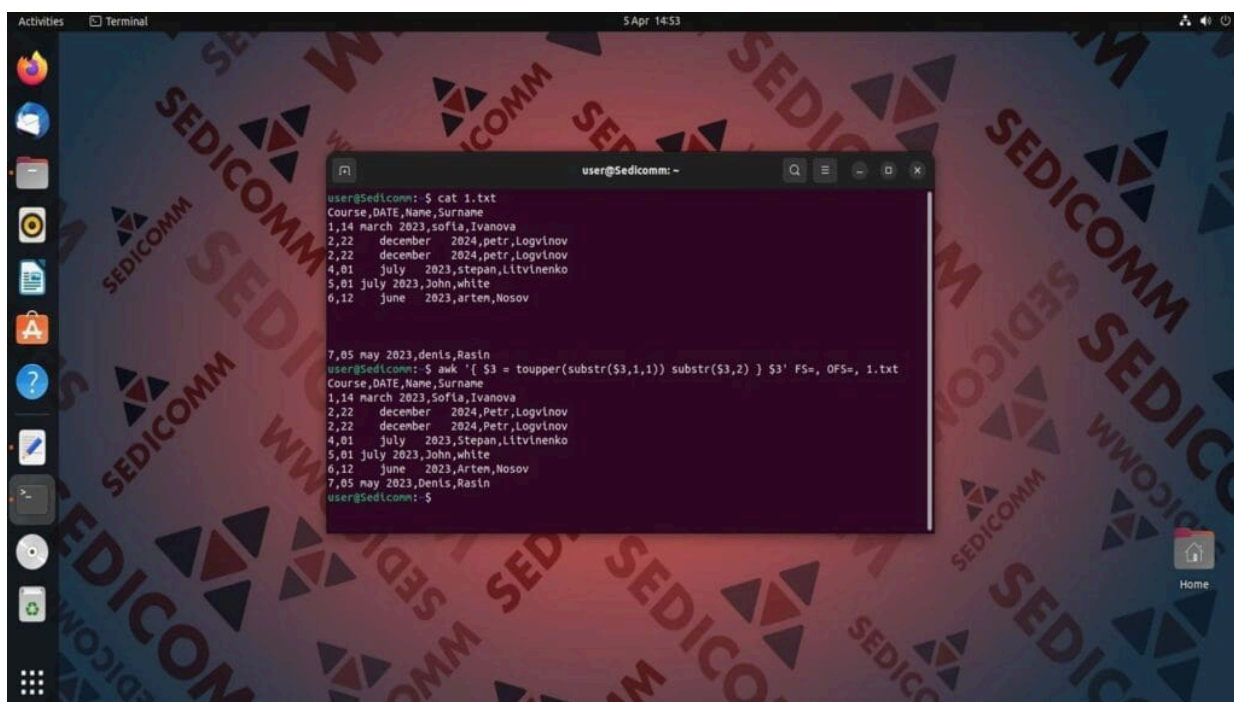
- `print` – выводит результаты работы команды на экран;
- `toupper($0)` – вызывает функцию преобразования из нижнего регистра в верхний с аргументом `$0`.

В итоге текст из файла будет преобразован в верхний регистр и выведен на экран.

Изменение части строки

Функции скриптового языка позволяют изменять заданные фрагменты текста в каждом поле. Для этого нам понадобится функция `substr(str, start, length)`. К примеру, преобразим в верхний регистр первую букву третьего поля каждой строки:

```
1 awk '{ $3 = toupper(substr($3,1,1)) substr($3,2) } $3' FS=, OFS=,
```



```
user@SedComm: ~$ cat 1.txt
Course,DATE,Name,Surname
1,14 march 2023,sofia,Ivanova
2,22 december 2024,petr,Logvinov
2,22 december 2024,petr,Logvinov
4,01 july 2023,stepan,Litvinenko
5,01 july 2023,John,white
6,12 june 2023,Arten,Nosov

7,05 may 2023,denis,Rasin
user@SedComm: ~$ awk '{ $3 = toupper(substr($3,1,1)) substr($3,2) } $3' FS=, OFS=, 1.txt
Course,DATE,Name,Surname
1,14 march 2023,Sofia,Ivanova
2,22 december 2024,Petr,Logvinov
2,22 december 2024,Petr,Logvinov
4,01 july 2023,Stepan,Litvinenko
5,01 july 2023,John,white
6,12 june 2023,Arten,Nosov
7,05 may 2023,Denis,Rasin
user@SedComm: ~$
```

Разберем эту конструкцию:



- `OFS=,` – устанавливает запятую (`,`) в качестве разделителя полей при выводе данных;
- `toupper(substr($3,1,1))` – переводит первую букву третьего поля в верхний регистр;
- `substr($3,2)` – извлекает оставшуюся часть третьего поля, начиная со второго символа;
- `$3 =` – присваивает третьему полю новое значение;
- `$3` – условие, проверяющее, что третье поле не пустое (это необходимо, чтобы избежать присвоения пустых значений полям).

Разделение полей на подполя

Одно поле строки может также содержать в себе несколько других элементов (например, `14 march 2023`). Следовательно, поля можно разбить на подполя. Например:

```
1 awk '+$1 { split($2, DATE, " "); print $1,$3, DATE[2], DATE[3] }'
```



```
user@Sedcomm: ~  
user@Sedcomm:~$ awk '+$1 { split($2, DATE, " "); print $1,$3, DATE[2], DATE[3] }' FS=, OFS=  
1.txt  
1,sofia,march,2023  
2,petr,december,2024  
4,stepan,july,2023  
5,John,july,2023  
6,arten,june,2023  
7,denis,may,2023  
user@Sedcomm:~$
```

Данный код производит такие действия:

1. `+$1` – проверяет, является ли первое поле числом, и если да, то выполняет следующую часть команды.
2. `split($2, DATE, " ")` – разбивает значения второго поля файла на подстроки, используя пробел в качестве разделителя. Результат разбиения будет сохранен в массиве `DATE`.
3. `print $1,$3, DATE[2], DATE[3]` – выводит значения первого поля, третьего поля, второй и третьей подстроки из массива `DATE`. Разделителем полей при выводе является запятая, что задается опцией `OFS=, .`

Использование внешних команд в AWK

В **AWK** можно вызывать внешние команды с помощью функции `system()`.

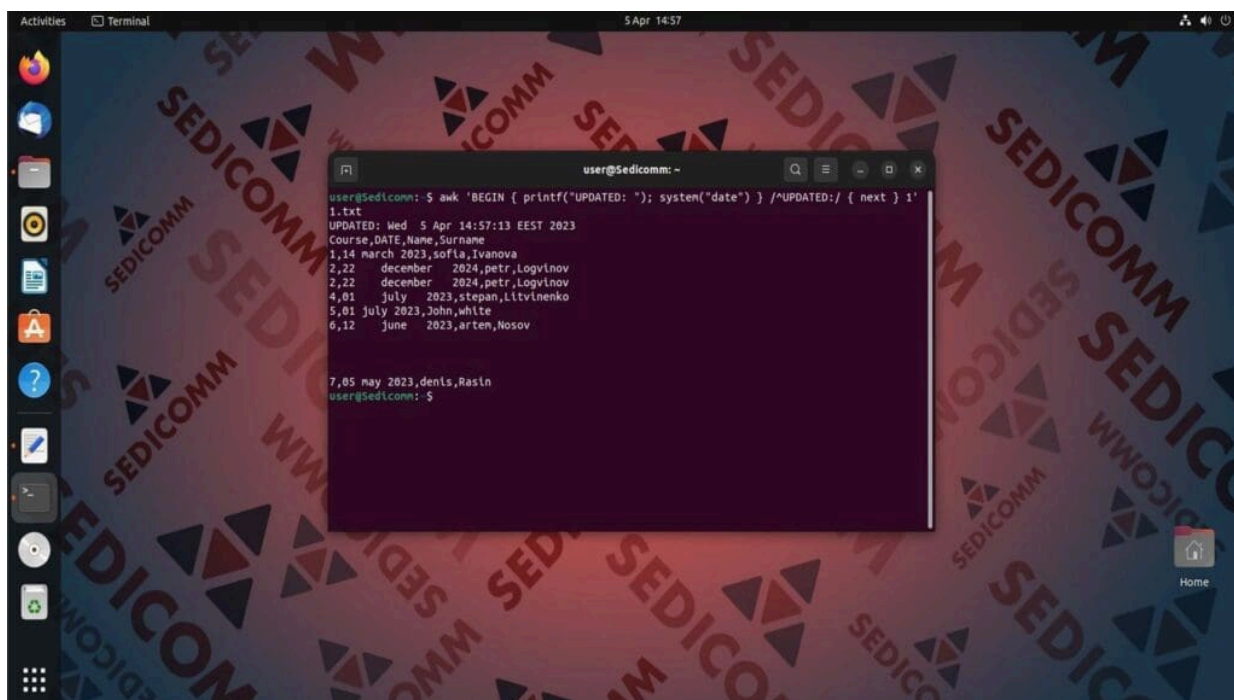
Синтаксис вызова команды следующий:

Здесь `"command arg1 arg2 argN"` – это строка, содержащая вызываемую команду и ее аргументы.

Добавление даты в файл

Например, добавим в первую строку текущее время системы:

```
1 awk 'BEGIN { printf("UPDATED: "); system("date") } /^UPDATED:/ {
```



Элементы этой конструкции выполняют определенные действия:

1. `BEGIN { printf("UPDATED: "); system("date") }` – выводит в терминал сообщение `UPDATED:` с помощью функции



2. `/^UPDATED:/ { next }` – проверяет, начинается ли текущая строка с сообщения `UPDATED:` . Если это так, то команда `next` пропускает строку и переходит к следующей.
3. `1` – соответствует любой строке, которая не была обработана ранее.

Изменение внешних переменных

Допустим, у Вас в таблице есть список пользователей системы и Вы хотите сгенерировать `UUID` и вставить его в новый столбец:

```
1 awk '+$1 { CMD | getline $5; close(CMD); print }' CMD="uuid -v4"
```

Этот код `AWK` использует внешнюю команду `uuid -v4` , чтобы сгенерировать `UUID` и затем вставляет его в пятое поле каждой строки входного файла.

Давайте более подробно рассмотрим каждую часть этой команды:

1. `CMD="uuid -v4"` – создает переменную `CMD` и устанавливает ее значение как строку `"uuid -v4"` . Эта переменная будет использоваться внутри `AWK` для запуска внешней команды.
2. `CMD | getline $5` – выполняет внешнюю команду, указанную в переменной `CMD` , и передает ее вывод в качестве ввода для `getline` .



4. `close(CMD)` – закрывает поток, созданный внешней командой.
5. `print` – выводит измененную строку на экран.

После выполнения скрипта каждая строка файла будет содержать уникальный **UUID**, сгенерированный внешней командой `uuid -v4`.

Вызов динамически сгенерированных команд

В **AWK** есть возможность динамически генерировать команды и выполнять их с помощью функции `system()`. Для этого необходимо создать строку, содержащую команду, и передать ее в функцию `system()`. Это может быть полезно, когда Вы хотите создать команду, используя значения из входного файла или переменных **AWK**.

Предположим, что у вас есть файл, содержащий названия файлов. Вы можете использовать **AWK** для создания и выполнения команды, которая выводит размер каждого файла в этом списке:

```
1 awk '{ cmd = "ls -l " $1 " | awk '{print $5}''; cmd | getline siz
```

В этом коде:

1. `cmd=` – присваивает переменной строчного типа значение, содержащее команду.



3. Затем скриптовый язык выполняет эту команду с помощью `cmd | getline size`.
4. `size` – это переменная **AWK**, которая будет содержать вывод команды.
5. Затем **AWK** закрывает команду с помощью `close(cmd)` и выводит имя файла и его размер.

Это простой пример того, как можно использовать динамически сгенерированные команды в **AWK**. Данный метод может быть особенно полезен, когда нужно обработать каждую строку отдельно.

Выводы

В этом руководстве мы рассмотрели основные сферы применения **AWK** и показали, как использовать его для выполнения различных задач **обработки текстовых файлов**. Мы начали с описания базовых элементов языка, таких как поля и операторы. А затем перешли к более продвинутым инструментам, таким как массивы, временные переменные и внешние команды.

Мы также рассмотрели различные примеры использования скриптового языка, включая форматирование вывода, обработку чисел с плавающей запятой, работу со строками и вызов внешних команд.

Спасибо за время, уделенное прочтению статьи!



ОБУЧЕНИЕ БЕСПЛАТНО

Новости ▾

Linux ▾

DevOps / DevNet 🔍 ▾

Подписывайтесь на обновления нашего блога и оставайтесь в курсе новостей мира инфокоммуникаций!

Чтобы знать больше и выделяться знаниями среди толпы IT-шников, записывайтесь на [курсы Cisco](#), [курсы по кибербезопасности](#), [полный курс по кибербезопасности](#), [курсы DevNet / DevOps \(программируемые системы\)](#) от Академии Cisco, [курсы Linux](#) от Linux Professional Institute на платформе SEDICOMM University (Университет СЭДИКОММ).

Курсы Cisco, Linux, кибербезопасность, DevOps / DevNet, Python с трудоустройством!

Спешите подать заявку! Группы стартуют 25 января, 26 февраля, 22 марта, 26 апреля, 24 мая, 21 июня, 26 июля, 23 августа, 20 сентября, 25 октября, 22 ноября, 20 декабря.

Что Вы получите?

- Поможем стать экспертом по сетевой инженерии, кибербезопасности, программируемым сетям и системам и получить международные сертификаты Cisco, Linux LPI, Python Institute.
- Предлагаем проверенную программу с лучшими учебниками от экспертов из Cisco Networking Academy, Linux Professional Institute и Python Institute, помощь сертифицированных инструкторов и личного куратора.



Как проходит обучение?

- Проведем вечерние онлайн-лекции на нашей платформе.
- Согласуем с вами удобное время для практик.
- Если хотите индивидуальный график – обсудим и реализуем.
- Личный куратор будет на связи, чтобы ответить на вопросы, проконсультировать и мотивировать придерживаться сроков сдачи экзаменов.
- Всем, кто боится потерять мотивацию и не закончить обучение, предложим общение с профессиональным **коучем**.

А еще поможем Вам:

- отредактировать или создать с нуля резюме;
 - подготовиться к техническим интервью;
 - подготовиться к конкурсу на понравившуюся вакансию;
 - устроиться на работу в Cisco по специальной программе.
- Наши студенты, которые уже работают там: жмите на **#НашиBCisco Вконтакте**, **#НашиBCisco Facebook**.

Чтобы учиться на курсах Cisco, Linux LPI, кибербезопасность, DevOps / DevNet, Python, подайте **заявку** или получите бесплатную **консультацию**.

Опубликовано: 16:42, 13 апреля, 2023. Обновлено: 16:56, 23 октября, 2024.

Рубрики: **Linux** Метки: **awk**, **linux**



ОБУЧЕНИЕ БЕСПЛАТНО

Новости ▾

Linux ▾

DevOps / DevNo 🔍 ▾

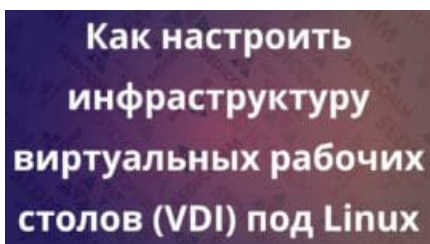
Артём Белкин

397 записей

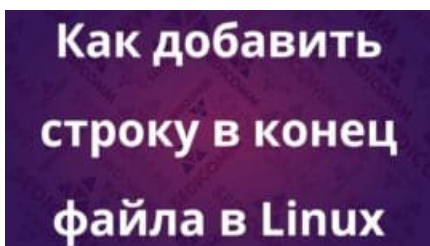
<https://edu-cisco.org>

Настоящий Linux-ниндзя, который живет в мире бесконечных терминалов и готов рассказать всем о преимуществах этой операционной системы, пока его еда не остынет!

Больше похожих постов



Как настроить
инфраструктуру
виртуальных рабочих
столов (VDI) под
Linux



Как добавить строку в
конец файла в Linux



ОБУЧЕНИЕ БЕСПЛАТНО

Новости ▾

Linux ▾

DevOps / DevNo 🔍 ▾

абсолютным
и относительным
путем в Linux

Разница между
абсолютным и
относительным путем в
Linux

Добавить комментарий

Ваш адрес email не будет опубликован. Обязательные поля помечены *

Комментарий *

Имя *

Email *

☐ Сохранить моё имя, email и адрес сайта в этом браузере для последующих моих комментариев.


CLOUDFLARE
[Конфиденциальность](#)
[Справка](#)



ОБУЧЕНИЕ БЕСПЛАТНО

Новости ▾

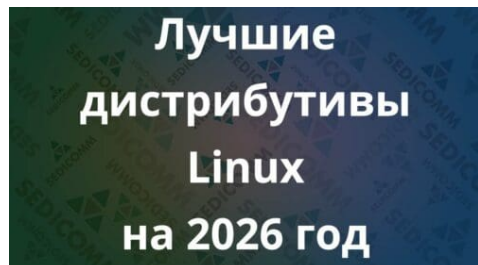
Linux ▾

DevOps / DevNo 🔍 ▾

Поиск по сайту



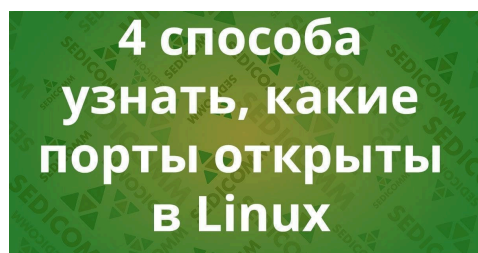
Лучшее



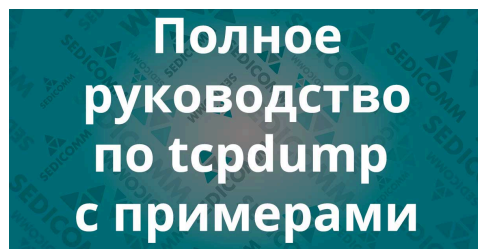
Лучшие дистрибутивы Linux на 2026 год



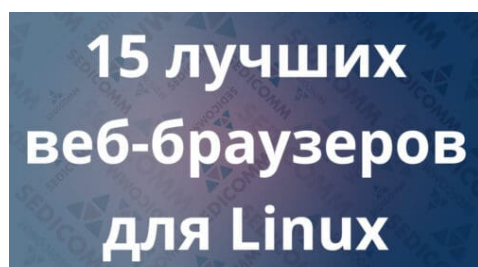
Как установить GoLang (язык программирования Go) в Linux



4 способа узнать, какие порты прослушиваются в Linux (открыты)



Полное руководство по tcpdump с примерами



15 лучших веб-браузеров для Linux



ОБУЧЕНИЕ БЕСПЛАТНО

Новости ▾

Linux ▾

DevOps / DevNo 🔍 ▾

параллельно с Windows 10
— подробная инструкция
для начинающих

подробная инструкция для начинающих

4 инструмента для мониторинга
температуры CPU и GPU в Ubuntu



4 инструмента для мониторинга
температуры CPU и GPU в Ubuntu

12 практических
примеров
команды grep
в Linux

12 практических примеров команды
grep в Linux

Полное
руководство
по команде
apt в Linux

Полное руководство по команде apt в
Linux

Поиск больших
файлов и папок
на диске
в Linux

Поиск больших файлов и папок на
диске в Linux

Популярное

Как настроить
инфраструктуру
виртуальных рабочих
столов (VDI) под Linux

Как настроить инфраструктуру
виртуальных рабочих столов (VDI) под
Linux



ОБУЧЕНИЕ БЕСПЛАТНО

Новости ▾

Linux ▾

DevOps / DevNo 🔍 ▾

строку в конец файла в Linux

Разница между
абсолютным
и относительным
путем в Linux

Разница между абсолютным и
относительным путем в Linux

Как посмотреть USB-устройства в ОС Linux

Как посмотреть USB-устройства в ОС
Linux

Лучшие
текстовые
веб-браузеры
в ОС Linux

Лучшие текстовые веб-браузеры в ОС
Linux

Три приложения
для создания
QR-кодов
в ОС Linux

Три приложения для создания QR-кодов
в ОС Linux

Команда chmod в Linux с примерами

Команда chmod в Linux с примерами

Как установить VLC Media Player в Ubuntu Linux

Как установить VLC Media Player в
Ubuntu Linux



ОБУЧЕНИЕ БЕСПЛАТНО

Новости ▾

Linux ▾

DevOps / DevNet 🔍 ▾

В Linux с примерами

Структура файла /etc/passwd в ОС Linux

Структура файла /etc/passwd в ОС
Linux

Рубрики

Новости

Новости IT

Linux ▾

Работа

Microsoft ▾

Статьи

DevOps / DevNet ▾

FreeBSD

Сети ▾

Mac OS

Свежие комментарии

Александр к записи [Как восстановить загрузку Cisco IOS
\(восстановление образа IOS\)](#)

Маша к записи [12 лучших self-hosted облачных хранилищ для
безопасного хранения ваших данных](#)



ОБУЧЕНИЕ БЕСПЛАТНО

Новости ▾

Linux ▾

DevOps / DevNo 🔍 ▾

Vt к записи [Rocket.Chat – бесплатный корпоративный чат с открытым исходным кодом для Linux](#)

Федор к записи [Использование скриптов expect в ОС Linux](#)

Федор к записи [Как установить Apache Maven в Ubuntu Linux](#)

algr14 к записи [Rsync \(Remote Sync\): 10 практических примеров использования команды Rsync в Linux](#)

Serg к записи [5 программ для сканирования документов в Linux](#)

Serg к записи [Два способа создать ISO из загрузочного USB в Linux](#)

Гость к записи [13 примеров команды cat для начинающих в Linux](#)

Последние записи

Разница между
абсолютным
и относительным
путем в Linux

Разница между абсолютным и относительным
путем в Linux

5 Июн в 13:25



60



0

Лучшие
текстовые
веб-браузеры
в ОС Linux

Лучшие текстовые веб-браузеры в ОС Linux

4 Июн в 11:36



129



0

Как вывести
список
последовательных
портов в Linux

Как вывести список последовательных
портов в Linux

28 Май в 12:06



248



0



ОБУЧЕНИЕ БЕСПЛАТНО

Новости ▾

Linux ▾

DevOps / DevNo 🔍 ▾

**в удобочитаемый
формат в Linux**

8 Май в 13:04

70 0

**3 способа
очистить
терминал в Linux****3 способа очистить терминал в Linux**

6 Май в 12:58

182 0

Свежие комментарии

Александр к записи Как восстановить загрузку Cisco IOS
(восстановление образа IOS)

Маша к записи 12 лучших self-hosted облачных хранилищ для
безопасного хранения ваших данных

Николай к записи 12 практических примеров команды grep в Linux

Vt к записи Rocket.Chat – бесплатный корпоративный чат с открытым
исходным кодом для Linux

Федор к записи Использование скриптов expect в ОС Linux

Федор к записи Как установить Apache Maven в Ubuntu Linux

algr14 к записи Rsync (Remote Sync): 10 практических примеров
использования команды Rsync в Linux

Serg к записи 5 программ для сканирования документов в Linux

Serg к записи Два способа создать ISO из загрузочного USB в Linux

Гость к записи 13 примеров команды cat для начинающих в Linux

Артем к записи Настройка сетевых соединений с помощью nmtui в Linux

Andrey к записи Как установить фреймворк Symfony в Linux

Популярные категории

Новости



ОБУЧЕНИЕ БЕСПЛАТНО

Новости ▾

Linux ▾

DevOps / DevNet 🔍 ▾

Linux

FreeBSD

Сети

MikroTik

Microsoft

DevOps / DevNet

Поиск...



[WPSM_AC id=27800]

© 2024 – blog.sedicom.com

[Блог SEDICOMM](#) [О сайте](#) [Университет](#) [Академия Cisco](#)
[Linux Professional Institute](#)